

User's Manual for
General Ocean Model
(GOM)

GOM_Rv1.2.0 (based on GOM_Dv90)

for

Any users who are interested in numerical modeling.

By

Jungwoo Lee

January 31, 2023

Preface

Why is it so complicated? Is there any possibility I can make it easy to use? These were two questions I had whenever I tried to learn a new numerical model; at least to me, it was not that easy to learn a new model. There is a proverb in Korea, “A thirsty man digs a well,” and I decided to develop an easy-to-use model by myself. I proposed a project “Development of the algal bloom forecasting model”, which aimed to develop a three-dimensional (3D) hydrodynamic model, right after I joined the Korea Institute of Civil Engineering and Building Technology (KICT) in 2015. I especially would like to thank Dr. Seog-Ku Kim, who was my supervisor at that time, for encouraging me to pursue the project even though most of the other modelers in Korea did not favor developing a new model but favored to use existing models. Even though I had experience with some well-known ocean circulation models, it was not easy to develop a model from the scratch in a few years. Luckily, I had my academic big brother Dr. Jun Lee; I and Dr. Jun Lee both graduated from the University of Florida. Dr. Jun Lee is a true expert in numerical modeling, and he had 3D model development experience. We pulled out his old source codes which were buried in an old USB drive, and we started to develop a new 3D model that aims to be an easy-to-use model.

My first numerical modeling experience goes back to 2003 in my master’s degree with the Environmental Fluid Dynamics Code which was developed by Dr. John Hamrick. Without any knowledge of numerical modeling or even without the knowledge of Fortran, I studied how to run the EFDC model by just reading the manual. Since 2003, I have experienced several other well-known ocean circulation models such as the Regional Ocean Modeling System (ROMS), the Finite Volume Community Ocean Model (FVCOM), Advanced Circulation Model (ADCIRC), and, recently, the Semi-Implicit Cross-Scale Hydrosience Integrated System Model (SCHISM). Each model has pros and cons, but overall, I loved the EFDC the most with its simplicity in the compiling process and user-friendly input and output file structures. For this reason, I benchmarked the input file structure of the EFDC when developing this model, the General Ocean Model (GOM). For this reason, users may find great similarities in input files between GOM and EFDC.

When we, I and Dr. Jun Lee, started to code this model, GOM, we aimed to achieve the following three main goals: (1) the model must be easy to use (i.e., user-friendly) even for beginners, (2) the model must be fast enough, and (3) the model must be stable and provide exact solutions. As I mentioned above, the first goal could be achieved by benchmarking the EFDC. To achieve the second and the third goal, we focused on the Casulli and Walters (2000) work. Following Casulli and Walters's (2000) paper, we could achieve the second and third goals. Now, at this point, I believe our model, GOM, fulfills the three proposed goals, thus I am proud of our model.

As one of the most ordinary modelers, I coded this model at the eye level of a common modeler. Thus, our model, GOM, might not have distinctive features as other models, which some genius modelers developed, have. However,

GOM might give users the most general functionalities which may most users ask the models. That is why I gave the name, the General Ocean Model, to our model. Also, the pronunciation of the GOM in Korean means “Bear,” and a bear is smart, strong, and even fast, and that is why I selected the name, GOM.

The journey to today, which I am writing this “preface”, was not that easy. While I was working in the KICT, I decided to move to the USA, and I quit the job in 2018. My first job since I moved to the USA was the St. Johns River Water Management District (SJRWMD). I worked at SJRWMD for eight months, and I loved my job at the SJRWMD, and I would like to thank my supervisor Dr. Peter Sucsy for his kind support. Then, I had to find (for some reason) the next job, and I joined the Texas Water Development Board in 2019. During this time, I had time to work on the GOM only at night and on weekends, since I had to do assigned works at the company. My lovely two sons, Jaden and Jasen, always asked me “Daddy, take a rest, otherwise, you will die.” Well, I have survived sons!

I spent a long time and a lot of effort learning what I now know about numerical methods including programming languages and Linux systems. Most of this learning process was quite painful at least to me. Because I know how painful this learning process is, I focused on the easiness of use when developing the GOM. I hope the GOM helps users to alleviate stresses as which I had when learning a new model, compiling source codes, and digging into some error sources. I can be reached at my email address at leejung24@ecu.edu or jungwoo33@gmail.com. Then, good luck with your journey to numerical modeling!

Jungwoo Lee
At my kids’ room, Austin, Texas
October 26, 2020

Acknowledgments

The initial development of the three-dimensional unstructured grid coastal and estuarine circulation model, which named the General Ocean Model (GOM), started in 2015 supported by funds from the Korea Institute of Civil Engineering and Building Technology (KICT) (project number: 20150121-0001-01), and the first deliverable stage was achieved with the support from the National Research Council of Science & Technology (NST) (No. CAP-18-07-KICT). The GOM source code has been developed led by Dr. Jungwoo Lee at the KICT in collaboration with Dr. Jun Lee at the Korea Coastal Ocean Modeling Systems (KCOMS).

GOM License Agreement

This License Agreement ("Agreement") governs the use, modification, and distribution of the General Ocean Model (GOM), developed by Dr. Jungwoo Lee and Dr. Jun Lee ("Original Developers"). By using this Software, you ("User") agree to the terms outlined below.

1. Ownership

- All intellectual property rights in GOM, including all modifications and updates, are and shall remain the exclusive property of the Original Developers, Dr. Jungwoo Lee and Dr. Jun Lee. All copyrights to the GOM code are reserved.

2. Permitted Use and Restrictions

- Users are granted permission to use the GOM for personal, academic, or non-commercial purposes.
- Users may modify and enhance the GOM under the condition that such modifications are implemented in collaboration with the Original Developers, as outlined in Section 3.
- The GOM may be used for commercial applications only with explicit prior written permission of Dr. Jungwoo Lee, the leader of the GOM development team.
- If a standalone software application or product is developed based on or derived from the GOM and is distributed or sold for commercial purposes, twenty percent (20%) of the net profit generated from such commercial use shall belong to the Original Developers, Dr. Jungwoo Lee and Dr. Jun Lee, unless otherwise agreed upon in writing.

3. Modification and Collaboration

- Users may propose modifications or updates to the GOM. All proposed changes must be submitted to and approved by Dr. Jungwoo Lee for integration into the original version of the Software. Once approved, the updated version may be officially redistributed.
- Users shall not independently distribute modified or updated versions of the GOM.
- Users agree to participate in the GOM community through the following mechanisms:
 1. Reporting code bugs and issues.
 2. Sharing major modifications made to the code.
 3. Contributing to discussions on model deficiencies, needed improvements, additions, and

successes.

- These mechanisms aim to benefit the entire GOM user community by facilitating the quick identification of issues, proposing solutions, and supporting the further development of the GOM source code and associated tools for processing, visualizing, and interpreting GOM model outputs.
- Users agree to acknowledge GOM in any publications resulting from its use and to refer to the model by its name, “GOM.”

4. Restrictions

- Redistribution of the GOM, whether in its original form or with modifications, is strictly prohibited without the explicit written permission of Dr. Jungwoo Lee.
- Users may not sublicense, rent, lease, or otherwise transfer rights to the GOM.

5. Contact Information

For questions, suggestions, or collaboration on updates, contact Dr. Jungwoo Lee:

- Email: leejung24@ecu.edu
- Personal Email: jungwoo33@gmail.com

6. Disclaimer

The GOM is provided "as is" without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. In no event shall the Original Developers be liable for any claims, damages, or other liabilities arising from the use of the GOM.

By using the GOM, you acknowledge that you have read, understood, and agreed to the terms and conditions of this License Agreement.

Table of Contents

Preface.....	i
Acknowledgments	iii
GOM License Agreement.....	iv
Table of Contents.....	vi
List of Figures	viii
List of Tables	x
1 Introduction.....	1
2 General Structure of the GOM.....	3
3 Essential Input Files.....	7
3.1 Grid Files (<i>cell.inp</i> & <i>node.inp</i>).....	8
3.2 GOM Master Input File (<i>main.inp</i>).....	8
4 Additional Input Files	41
4.1 harmonic_ser.inp.....	41
4.2 eta_ser.inp	42
4.3 q_ser.inp.....	43
4.4 wind_ser.inp.....	44
4.5 hurricane_ser.inp.....	45
4.6 salt_init.inp & salt_ser.inp / temp_init.inp & temp_ser.inp	46
4.7 Obc_info.inp & Qbc_info.inp.....	47
5 Downloading, Compiling, and Executing the Code	48
5.1 Install Fortran Compiler & NetCDF library	48
5.1.1 Linux & Unix.....	48
5.1.2 Windows	50

5.1.3	MSYS2 install guide.....	50
5.2	Compile Source codes	53
6	Source Codes and Subroutines	55
7	References.....	56

List of Figures

Figure 3-1. File cell.inp.	8
Figure 3-2. File node.inp.	8
Figure 3-3. Card Image C00.	10
Figure 3-4. Card Image C01.	10
Figure 3-5. Card Image C1.	10
Figure 3-6. Card Image C2 & C2_1.	12
Figure 3-7. Schematic diagram of the z-grid system used in C2 & C2_1.	13
Figure 3-8. Card Image C3.	14
Figure 3-9. Card Image C4.	15
Figure 3-10. Card Image C5_01 ~ C5_04.	16
Figure 3-11. Card Image C6.	18
Figure 3-12. Card Image C7.	19
Figure 3-13. Card Image C8.	21
Figure 3-14. Card Images C20 & C20_1.	22
Figure 3-15. Card Image C21.	24
Figure 3-16. Card Image C22 & C22_1.	25
Figure 3-17. Card Image C23 & C23_1.	26
Figure 3-18. Card Image C30.	29
Figure 3-19. Card Image C31 and C31_1.	30
Figure 3-20. Card Image C32.	31
Figure 3-21. Card Image C40.	32
Figure 3-22. Card Image C41, C41_1, & C41_2.	33
Figure 3-23. Card Image C42 & C42_1.	34

Figure 3-24. Card Image C43 & C43_1	36
Figure 3-25. Card Image C44 & C45	37
Figure 3-26. Card Image C46, C46_1, C46_2, & C46_3	38
Figure 3-27. Card Image C47	40
Figure 4-1. harmonic_ser.inp	41
Figure 4-2. eta_ser.inp	42
Figure 4-3. q_ser.inp	43
Figure 4-4. wind_ser.inp	44
Figure 4-5. hurricane_ser.inp	45
Figure 4-6. salt_init.inp	46
Figure 4-7. salt_ser.inp	46
Figure 4-8. Obc_info.inp	47
Figure 5-1. gfortran version check result	49
Figure 5-2. NetCDF install check	49
Figure 5-3. NetCDF library install check	49
Figure 5-4. NetCDF Fortran library install check	50
Figure 5-5 NetCDF Fortran library install check on MSYS2 Mingw64	53
Figure 5-6. NetCDF library path update in the Makefile	53

List of Tables

Table 2.1. Input files for the GOM.	3
Table 2.2. Output files for the GOM.	4
Table 6.1. Subroutines in GOM. (not yet finished)	55

1 Introduction

A three-dimensional unstructured grid finite-volume model for coastal and estuarine circulation, which I (Jungwoo Lee) named the General Ocean Model (GOM), has been developed. Combining the finite volume and finite difference methods, GOM achieved both the exact conservation and computational efficiency. The propagation term was implemented by a semi-implicit numerical scheme, so-called θ scheme, and the time-explicit Eulerian-Lagrangian Method (ELM) was used to discretize the nonlinear advection term to remove the major simulation limitations of the time step, which appears when solving shallow water equations, by the Courant-Friedrichs-Lewy stability condition. Because the GOM uses orthogonal unstructured computational grids, allowing both triangular and quadrilateral grids, considerable flexibility to resolve complex coastal boundaries is allowed without any transformation of governing equations. More fundamental details of the GOM can be found in the original development paper (Lee et al., 2020), thus details of the governing equations and the solution algorithms, which used in GOM, are not duplicated in this manual.

The general organization of this manual is as follows. Chapter 2 presents the general structures of the GOM. Chapter 3 describes essential input files including the master input file, *main.inp*, which controls the overall execution of a model simulation. Chapter 4 explains additional input files. Chapter 5 explains how to compile and make an executable file from the source codes. A list of subroutines and a brief description of their functions are presented in Chapter 6.

The followings are the additional notes you may need to understand when reading this manual and GOM source code:

- GOM is originally coded in standard FORTRAN 90 and tested with GNU Fortran (64 bit) both on Windows and Linux systems.
- The released version will have the following format (e.g.):
 - GOM_Rv1.0.0 (based on GOM_Dv84)
 - Here, the first two characters after “GOM_”, “R” and “v” denotes the “Released” and “version”.
 - Then the first number after “v”, i.e., 1 in this example, is the major version number. If there is a major update in source codes, this number will be updated. The second number, 0 in this example, will be updated only if input file formats are changed from the previous version. The third number, 0 in this example, will be updated when any parts of source codes are updated (typically it happens during the debugging process).
 - The term “(based on GOM_Dv**)” denotes the corresponding “Developer” version (i.e., the version on my side as a model developer).
- Most of my personal comments in source codes which I included during the model development are

intentionally deleted in the released version of the source codes with the line containing “jw”. Thus, if you want to have the original developer’s version including either nice or dirty comment lines, contact me (leejung24@ecu.edu).

- Throughout this manual, the folder names are denoted with “***bold & Italic***” text, and the variable names are denoted with “**bold**” text.

2 General Structure of the GOM

The current version of the GOM (GOM_Rv1.0.0) comprises more than one hundred standard FORTRAN 90 source codes including one main program (*main.f90*) and 4 module files. Global variables are defined in *mod_global_variables.f90*, and all of the names of input files and output files are defined in *mod_file_definition.f90*, and you can easily modify the input or output file names and folders in which output files will be stored during the simulation. A general flow structure of the GOM is coded in the main source code, *main.f90*; you can find brief descriptions of source codes in the GOM in Section 6.

In the current version of the GOM (version 1.0.0), three essential input files are required, and you can specify additional input files for their simulation purpose. I will explain with the following example project folder structure:

- project_folder (this is the name of your project folder)
 - input (all input files are in this folder)
 - output (all output files will be in this folder)
 - run.exe (this is the binary executable generated from the source code folder)

All input files have an extension with ‘*.inp’, and they should be in the ‘*input*’ folder, which should be located under the same folder the executable file (e.g., *run.exe*) is located in. Note that you can change this folder location and name easily in *mod_file_definition.f90* as your preference. A brief description of input files is presented in Table 2.1.

Table 2.1. Input files for the GOM.

Group	File Name	Description
Essential files	main.inp	Master input file.
	cell.inp	Horizontal cell (grid) information file. It includes a total cell number and cell connectivity.
	node.inp	Horizontal node information file. It includes a total node number and x and y coordinates of each node.
Optional input files	eta_ser.inp	Open boundary water surface elevation (η) time series file.
	harmonic_ser.inp	Open boundary tidal (harmonic) elevation time series file.
	q_ser.inp	River discharge (Q) time series file.
	windp_ser.inp	Wind and air pressure time series file.
	hurricane_ser.inp	Wind and air pressure time series at each node (another form of windp_ser.inp).

hurricane_center.inp	Hurricane center information; it is required if hurricane simulation is on.
bottom_roughness.inp	Bottom roughness at each node; use if the spatially varying bottom friction option, bf_varying , is selected.
salt.inp	Initial salinity distribution at each node.
salt_ser.inp	Salinity time series file.
temp.inp	Initial temperature distribution at each node.
temp_ser.inp	Temperature time series file.
Obc_info.inp	The open boundary information file
Qbc_info.inp	River boundary information file

Simple test cases without open boundaries can be simulated only with three essential files: *main.inp*, *cell.inp*, and *node.inp*. If open boundaries are forced by harmonic tides (tidal boundary), you should provide *harmonic_ser.inp*, which includes a series of tidal periods, tidal amplitudes, and tidal phases. If open boundaries are forced by the measured surface elevation time series data, the information can be provided with *eta_ser.inp*. If rivers are described in the model grid system, the river discharge time series should be provided with *q_ser.inp*. Wind information can be applied either with *windp_ser.inp* or *hurricane_ser.inp*.

Most output files have an extension with ‘*.dat’, and they will be automatically located in the ‘output’ folder, which should be located under the same folder the executable file (*run.exe*) is located in. A brief descriptions of output files are presented in Table 2.2.

Table 2.2. Output files for the GOM.

Group	File Name	Description
Check files	run.log	The log file for the model simulation.
	main_mirr.dat	The mirror image file for <i>main.inp</i> . It will be automatically generated while reading <i>main.inp</i> . So, you must see this file whenever you run a model.
	cell_mirr.dat	The mirror image file for <i>cell.inp</i> . It will be generated while reading <i>cell.inp</i> if cell_mirr is set to 1 in <i>main.inp</i> .
	node_mirr.dat	The mirror image file for <i>node.inp</i> . It will be generated while reading <i>node.inp</i> if node_mirr is set to 1 in <i>main.inp</i> .

	dia_check_geometry.dat	Model grid information file. It will be automatically generated while reading input files. It contains grid-related information, so you can diagnose your model grid.
	check_grid_2D.dat (vtk)	2D horizontal model grid file; origin shifted version. It will be generated if check_grid_2D is activated.
	check_grid_2DO.dat (vtk)	2D horizontal model grid file; original version. It will be generated if check_grid_2DO is activated.
	check_grid_3D.dat (vtk)	3D model grid file. It will be generated if check_grid_3D is activated.
	voronoi_center.dat	Voronoi center information at each element. It will be generated if voronoi is activated.
	voronoi_face_center.dat	Voronoi face center information. It will be generated if voronoi is activated.
	check_eta.dat	Water surface elevation checking file for the given water surface elevation time series (<i>eta_ser.inp</i>). It will be generated if check_eta is set to 1. It will show the interpolation results every 30 minutes. Not yet implemented.
	check_Q.dat	Given river discharge time series (<i>q_ser.inp</i>) checking file. It will be generated if check_Q is set to 1. It will show the interpolation results every 30 minutes.
Simulation result files		
	eta_tser_from_msl.dat (txt)	Water surface elevation time series file. It will be generated if tser_eta is on.
	H_tser.dat (txt)	Total water depth time series file. It will be generated if tser_H is on.
	u_tser.dat (txt)	True-east velocity time series file. It will be generated if tser_u is on.
	v_tser.dat (txt)	True-north velocity time series file. It will be generated if tser_v is on.
	salt_tser.dat (txt)	Salinity time series file. It will be generated if tser_salt is on.
	temp_tser.dat (txt)	Temperature time series file. It will be generated if tser_temp is on.
	airp_tser.dat (txt)	Atmospheric pressure time series file. It will be generated if tser_pressure is on.

	tec2D_#.dat vtk2D_#.txt	2D horizontal time series file for selected variables. It will be generated if IS2D_switch is on.
	tec3D_full_#.dat vtk3D_full_#.txt	3D horizontal time series file for grid only. It will be generated if IS3D_full_switch is on.
	tec3D_surf_#.dat	3D horizontal time series file for the surface only. It will be generated if IS3D_surf_switch is on.
	tec2D_flood_map.dat vtk2D_flood_map.txt	2D flood map file. It will be generated if IS2D_flood_map is on.
	dump2D_#.dat	2D dump file. It will be generated if IS2D_dump_switch is on.
	dump3D_#.dat	3D dump file. It will be generated if IS3D_dump_switch is on.
	netcdf_#.nc	NetCDF out file It will be generated if netcdf_switch is on.
	dia_nonlinear_advection.dat dia_momentum_equation.dat dia_freesurface_equation.dat dia_eta_at_ob.dat dia_bottom_friction.dat dia_face_velocity_uv.dat dia_face_velocity_w.dat dia_node_velocity.dat	Diagnostic files for some important subroutines.

3 Essential Input Files

The GOM requires three essential input files: *cell.inp*, *node.inp*, and *main.inp*. First, you should prepare geometric grid information files, and both unstructured triangular and rectangular grids or combined triangular/rectangular grids can be used in the GOM. You can use any commercial or non-commercial unstructured grid generation programs. After preparing the two geometric information files, you should provide the master input file, *main.inp*.

All input files can be started with header lines which start either with one of “!”, “C”, or “c”, at the first column of the line; i.e., If a line starts with these characters, the line will be treated as a header line (or comment line), and it will not be read. There is no limit for this, i.e., you can insert or exclude header lines as many as you want, and it depends on the your preference. I believe this approach gives you much flexibility when handling input files. **Note: If space exists before these comment characters, the line will not be treated as a comment line, and the reading process will be terminated.**

However, there is one important thing you need to keep in mind when dealing with *main.inp*. Even though “C” or “c” which appears in the first column of a line is treated as a comment line, the card number should not be changed, so use them as given; It is because the card number is used as identification when finding each card information. Note that I use the term “Card” when explaining the input file, and I borrowed the term from a “punch card” (see more details here: https://en.wikipedia.org/wiki/Computer_programming_in_the_punched_card_era).

Some card images have names with an underbar, then this card is, mostly, a child card; e.g., C2_1 is a child card of C2. In this case, this child card depends on the parent card, and you must carefully turn on/off this child card. However, some cards are independent cards even though it contains an underbar, e.g., C5_01 ~ C5_04. You can easily distinguish these independent cards from dependent child cards since these independent card starts with “0” at the right sub-number (i.e., C5_01 not C5_1 in this example).

One more important thing, which may be worthwhile to mention especially for those who are not familiar with Fortran, is the variable name. Most of the variable names in GOM are shown in lower cases since it is my (Jungwoo Lee) preference as the main model developer, however, you should know that Fortran does not distinguish lower or upper cases. Thus, the following variables are identical in Fortran; for example, ‘MAXELE’, ‘Maxele’, ‘maxele’, or even ‘maxELE’ are the identical variables in GOM.

In the following subsections, I will show an example template figure first, then some explanations will be followed. In most cases, you may be able to get a clear idea by just reading each card image, thus I will only explain something additional only if it requires.

3.1 Grid Files (*cell.inp* & *node.inp*)

```
! cell.inp:
! If you want to add more comments, then put '!' at the first column of the line.
!
! cell_num
! ELEMENT(I), Tri(=3)/Quad(=4), (NODE(I,K),K=1,Tri/Quad)
!
35127
1 3 2 1 3
2 3 2 6 1
3 3 1 6 5
.
.
.
35125 3 18717 18714 18715
35126 3 18716 18714 18717
35127 3 18717 18718 18716
```

Figure 3-1. File cell.inp.

The *cell.inp* starts with the total cell number, and rows of the total cell number follow (Figure 3-1). Each row consists of either 5 or 6 columns depending on each element's geometry, i.e., triangular, or quadrilateral element. The 1st column is the element's ID number; the 2nd column is the element's shape type defined as 3 (for triangular) or 4 (for quadrilateral); the 3rd ~ 5th or 6th columns are node numbers that construct the element.

```
!C node.inp:
!C If you want to add more comments, then put '!' at the first column of the line.
!C
!C totla_node_number
!C node_num      x[m]          y[m]          depth[m]
!C
18718
1 0.4053641000E+06 0.3431161830E+07 0.555000E+01
2 0.4049900000E+06 0.3430973000E+07 0.555000E+01
3 0.4048630000E+06 0.3431330000E+07 0.555000E+01
.
.
.
18716 0.3555866520E+06 0.3354712700E+07 0.115140E+01
18717 0.3560890000E+06 0.3355172000E+07 0.797700E+00
18718 0.3555860000E+06 0.3355225000E+07 0.993100E+00
```

Figure 3-2. File node.inp.

Similarly, the *node.inp* starts with the total node number, then rows of the total node number follow (Figure 3-2). The 1st column is the node ID number; the 2nd and 3rd columns are x- and y-coordinates in meters, respectively; the 4th line is the water depth (positive for a wet (i.e., water) element and negative for a dry (i.e., land) element).

3.2 GOM Master Input File (*main.inp*)

The master input file, *main.inp*, consists of several card images. The file, *main.inp* is self-documented with definitions and guidance for each input variable contained in its card image section. Description lines start with an exclamation mark, “!”, thus you can freely insert more information lines or delete lines. Description parts in each card image section include variable names and an explanation of the variable. In addition, a type of variable is shown in square brackets after the name of each variable. For example, [c] represents the ‘character’ type of a

variable; [i] represents the ‘integer’ type of a variable; and [r] represents the ‘real’ type of a variable. I think this will help you to understand what kinds of data types are used in GOM.

Note, as I mentioned previously, that there is one more comment method used mostly in *main.inp*. A line starts with “C” or “c” will be treated as a comment line as well as “!”. One more trick is used here. Each card number must start either with “C” or “c” even though it is treated as a comment line. With this trick, you can insert as many comment lines, or you can even delete all the comment lines except the first card number line. I think this trick provides you with a lot of freedom when using this model.

The *main.inp* consists of the following four card sections:

- C0 ~ C19: General Model Configuration Information
 - C8 is the end of this set
- C20 ~ C29: Tidal & River Boundary Conditions
 - C25_1 is the end of this set
- C30 ~ C39: Air/Sea Boundary Conditions
 - C32 is the end of this set
- C40 ~ C49: Output Control Options
 - C46 is the end of this set

As you can see here, there are some card number gaps between each section, and there is the reason why I designed *main.inp*. Those gaps are the placeholders for the future options that may need to be included as model upgrading.

Now, let’s start taking a look at each card; note the card images used here are from the Mobile Bay test case.

```

=====
!
! Main input file for the General Ocean Model (GOM)
!
! Welcome to the GOM model
! Developed by Jungwoo Lee & Jun Lee
! This file is self-documented with definitions and guidance for each
! input variable contained in its card image section.
!
=====
! C0 ~ C19: General Model Configuration Information
!
=====
C00  TITLE FOR RUN
!
!   project_name:  [c]   TITLE OR IDENTIFIER FOR THIS INPUT FILE AND RUN.
!                   (LIMIT TO 200 CHARACTERS LENGTH)
!
C00  project_name
      Mobile_Bay:GOM_v84_synchronized
=====

```

Figure 3-3. Card Image C00.

```

=====
C01  Define some global parameters before we start
!    Generally, users don't need to change these values.
!
!    gravity       [r]    Gravitational acceleration, g [m/s2]
!    rho_o         [r]    Reference density of seawater [kg/m3]
!    rho_a         [r]    Reference density of air [kg/m3] @ 0 degree Celsius
!    max_no_neighbor_node [i] Maximum allowed number of neighbor nodes and elements in your grid
!                           Typically, 10 is o.k. for general grid.
!
C01  gravity      rho_o      rho_a      max_no_neighbor_node
      9.81        1025.0    1.29      10
=====

```

Figure 3-4. Card Image C01.

```

=====
C1   Read coordinate frame flag and check it
!
!    maxnod        [i]    Number of nodes in your grid
!    maxele        [i]    Number of elements in your grid
!    h_node_adjust [r]    Additive adjustment to the node depth provided in "node.inp".
!                           This is an easy grid depth adjustment option.
!    ana_depth     [i]    0/1 off/on, overwrite water depth with analytical initial conditions
!                           If 1, re-define water depth in "ana_depth.f90".
!                           In this case, water depth defined in node.inp will be replaced with the given analytic informati
!
!    coordinate_system [i] 1 Cartesian or UTM system
!                           2 Longitude/Latitude system. Lon. and lat. should be in [degree]
!    lon           [r]    Longitude of the model domain [degree]; this will be used for heat transport
!    lat           [r]    Latitude of the model domain [degree]; this will be used for heat transport & Coriolis calculation
!    voronoi       [i]    0/1 off/on, use voronoi center of elements, otherwise Centroid of a triangle will be used.
!    node_mirr:    [i]    0/1 off/on, Write mirror image file (node_mirr.dat)
!    cell_mirr:    [i]    0/1 off/on, Write mirror image file (cell_mirr.dat)
!
C1   maxnod  maxele  h_node_adjust  ana_depth  coordinate_system  lon  lat  voronoi  node_mirr  cell_mirr
      18718   35127   0.0            0          1             -128.0  37.5  0        1          1
=====

```

Figure 3-5. Card Image C1.

You can identify the project title in Card Image C00 (Figure 3-3); note that this information does not affect the model simulation at all. Card Image C01 (Figure 3-4) includes some basic global variables, and you may not need to change these values.

Card Image C1 (Figure 3-5) consists of a grid and coordinate-related variables. The **maxnod** and **maxele** are the total node and element numbers in the grid, respectively, and these numbers must be identical to the total node number and element number provided in **node.inp** and **cell.inp**, respectively. If the **ana_depth** is on, then you can define water depth analytically, overwriting the provided depth information in **node.inp**, in the **ana_initial.f90**, then

you must compile source codes again and build a new executable. This option is specifically useful when you want to test the water depth effect while fixing the horizontal grid.

The **coordinate_system** defines the model grid unit system in *node.inp*, i.e., if the **coordinate_system** == 1, the information in *node.inp* should be in meter. If the **coordinate_system** == 2, the grid information in the *node.inp* should be in longitude and latitude degrees. The variables **lon** and **lat** define the approximate center coordinates of the model grid, and they will be used to calculate the Coriolis parameter and in heat transport calculation.

The **Voronoi** is the off/on option for the Voronoi center calculation. GOM uses the unstructured orthogonal grid, and it requires some restrictions when generating grids. Thus, GOM allows using a less restrictive geographical limit as using the centroid of an element rather than using a strict Voronoi center. Thus, you can try to turn on/off this **Voronoi** option. If GOM fails with the **Voronoi** option, turn off this option.

When using a numerical model, most of the errors are associated with the main input files, and sometimes it is quite difficult to find the source of the error. For this reason, GOM generates mirror image files when reading *main.inp*, *node.inp*, and *cell.inp*, to make it easy to check errors associated with major input files. The main input file's mirror image file, *main_mirr.dat* will be generated automatically, but the other two files have a turning on/off option here.

```

=====!
C2   Read vertical layer information
!
!   maxlayer      [i]   Number of vertical layers
!   MSL           [r]   Mean Sea Level elevation from the bottom (z_level(0)) [m]
!   z_level(0)    [r]   Reference bottom level elevation below MSL [m]
!
C2   maxlayer      MSL      z_level(0)
      5            0.0     -100.0
=====!
C2_1 [maxlayer] lines are required
!
!   serial_num    [i]   Layer number (dummy number)
!   z_level       [r]   each vertical grid level [m] (each grid's top elevation) from the bottom (i.e., z_level(0))
!                       Provide this in surface-to-bottom order.
!
C2_1 serial_num    z_level(i)
      5            20.0      ! 5 layers
      4            -3.0
      3            -6.0
      2            -9.0
      1            -12.0
=====!

```

Figure 3-6. Card Image C2 & C2_1.

Card Images C2 and C2_1 (Figure 3-6) are about the vertical layer information. The current version of GOM uses the z-grid system, and the **maxlayer** defines the number of vertical z layers; the vertical layer number starts from the bottom. There is another variable that defines the z-grid system in GOM, **z_level**. Each vertical layer has top and bottom faces, and they are defined as, assuming the k th vertical layer, **z_level(k)** for the top face, and **z_level(k-1)** for the bottom face (Figure 3-7).

The **MSL** is the mean sea level elevation from the arbitrary bottom level (**z_level(0)**), and the **MSL** must be located above the following **z_level(0)**. For example, let's assume the maximum depth in the model grid is 50.0 m, and you want to set the mean sea level at $z = 0.0$ m. Then, **z_level(0)** must be equal to or less than -50.0 m; i.e., **z_level(0)** can be any number below -50.0 m (in Figure 3-7, I used **z_level(0) = -100.0** m). In other words, if you want to set the arbitrary bottom at $z = 0.0$ m, then the **MSL** can be located at any elevation greater than $z = 50.0$ m. So, it depends on your preference. Figure 3-7 shows details of the z-grid system used in GOM.

The C2_1 is a child card of the C2, and it requires **maxlayer** lines. You must provide vertical layer information from the surface to the bottom order as shown in Figure 3-6. The **serial_num** is the vertical layer number, and it starts from the surface layer. Thus, for example, if the **maxlayer** is 5, this variable starts from 5 to 1. However, it is a dummy number, so GOM will not use this number as given but use it as its order.

With the given information in C2 and C2_1, GOM will calculate **delta_z**, a vertical layer space in [m], which is defined as $\text{delta_z}(k) = \text{z_level}(k) - \text{z_level}(k-1)$, where **z_level(k)** and **z_level(k-1)** are the vertical locations of the top and bottom faces of the k th layer, respectively (see Figure 3-7). One important thing that you must understand is the surface layer's top level, i.e., **z_level(maxlayer)**, should be defined sufficiently above the **MSL** to capture the flooding event. If the top level is defined as close to the **MSL**, and if the predicted (simulated) water surface elevation is greater than the given top level, GOM will be terminated. If the model domain has initially dry grids (i.e., bottom elevation is greater than the mean sea level), the value of **z_level(maxlayer)** must be greater than the

highest bottom elevation in entire model domain.

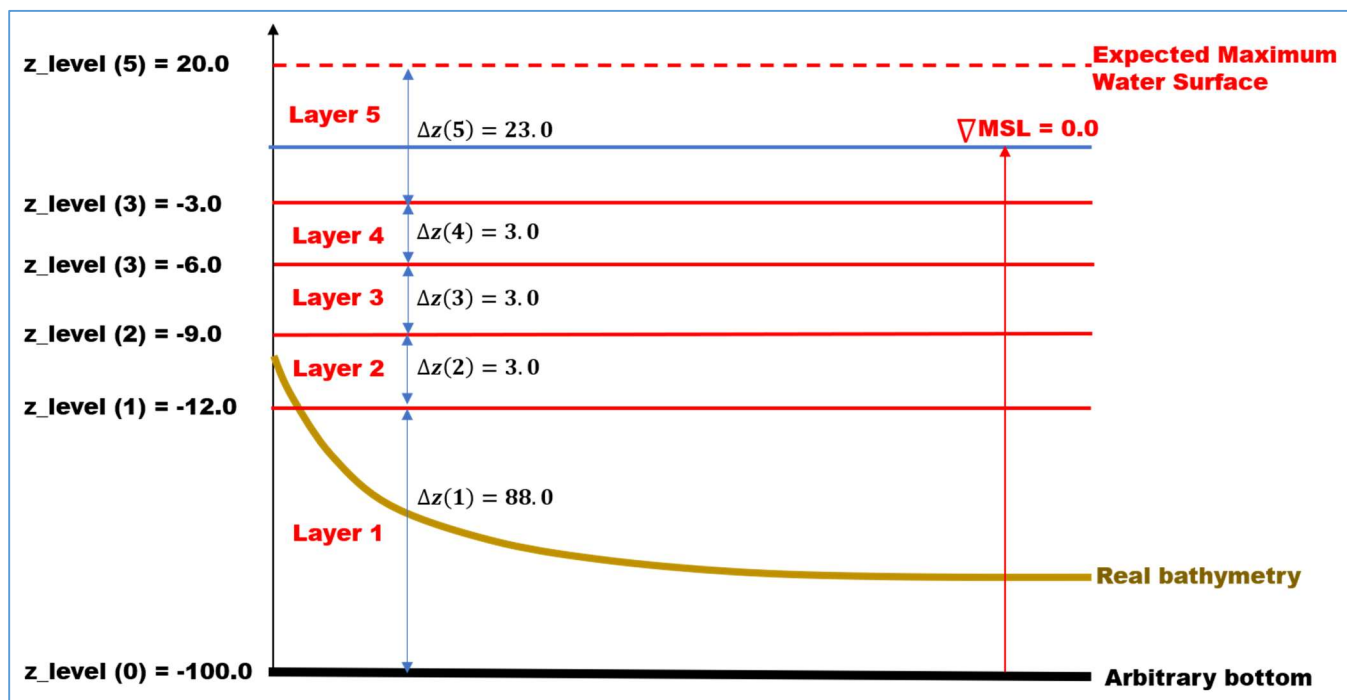


Figure 3-7. Schematic diagram of the z-grid system used in C2 & C2_1.


```

=====!
C3   Read time-related variables
!
!   dt           [r]   Time increment in [sec]
!   ndt          [i]   Total simulation step
!                       Total simulation time = dt * ndt [sec]
!   data_start_year [i] All given input data's start (reference) year (except hurricane_ser.inp)
!                       All data starts at data_start_year/January/1st 00:00:00
!   data_time_shift [i] Data start time shift in [day] from data_start_year/January/1st 00:00:00
!   start_year      [i] Simulation start year. Should be 1900 <= start_year <= 2099
!   start_month     [i] Simulation start month
!   start_day       [i] Simulation start day
!   start_hour      [i] Simulation start hour
!   start_minute    [i] Simulation start minute
!   start_second    [i] Simulation start second
!
C3   dt      ndt   data_start_year  data_time_shift  start_year  start_month  start_day  start_hour  start_minute  start_second
!   60.0    43200 2010             212             2010      8           1         0           0           0
!   90.0    28800 2010             212             2010      8           1         0           0           0
!   150.0   17280 2010             212             2010      8           1         0           0           0
!   300.0   8640 2010             212             2010      8           1         0           0           0
=====!

```

Figure 3-8. Card Image C3.

Card Image C3 (Figure 3-8) consists of a line with simulation time-related variables. It might be a bit confusing for a new user, however, this approach will give you much more flexibility when developing a model input data. Once model input data is generated, you can simulate GOM from any data time range. For example, if the provided input files (such as boundary elevation and river discharge data) start from January 1st, 2000 and you want to simulate from March 15, 2000, you don't need to correct (or change) model input data but just run the model from March 15, 2000, with the following settings:

- **data_start_year** = 2000
- **data_time_shift** = 0
- **start_year** = 2000
- **start_month** = 3
- **start_day** = 15
- **start_hour, start_minute, start_second** = 0

It is also easy to accumulate model input data for a long period, and you just need to add more data at the end of the existing input file. The **data_time_shift** is also useful when there is a mistake (or time just starts from 0.0 in any month) in the generated input files; you can easily shift the data's beginning time. Note that there is one more time shift option in each time series input file, and it will be explained later. The example in Figure 3-8 shows that the model simulation starts from 08/01/2010, and the provided data accumulated also from 08/01/2010 00:00:00 setting **data_start_year** = 2010 and **data_time_shift** = 212; this “212” is the Julian date at July 31, 2010; i.e., August 1, 2010 starts from 212 days after January 1, 2010.

As mentioned previously (in Section 3.2), you can include multiple comment lines starting with “C” or “c” and “!”. Thus, as shown in Figure 3-8 you can keep several different time step settings (i.e., 60.0, 90.0, and 300.0 seconds) for the next use; in this example, the simulation time step (Δt) is set to 150 seconds. This option allows you much freedom to easily switch model setups with different configurations.

```

=====!
C4   Restart & screen show control
!
!   restart_in:      [i]   0   Cold start: normal case
!                       1   Hot start: 'restart.inp' is required.
!   restart_out:     [i]   0   Do not write restart file (restart_***.out)
!                       -1  Write restart file (restart_***.out) once at the end of run
!                       1   Write restart file at every restart_freq
!   restart_freq:    [i]   N   Write restart_***.out every restart_freq period (N * dt seconds)
!   ishow:           [i]   0   Do not show run-time results on screen
!                       1   Show run-time results on screen (full show)
!                       2   Show only beginning & ending on screen (skip showing run-time results)
!   ishow_frequency: [i]   N   Screen show time interval (N * dt seconds)
!
!
C4   restart_in restart_out restart_freq ishow ishow_frequency
!   0           0           14400       1      60                ! every hr; dt = 60, every 10 day
!   0           0           28800       1      40                ! every hr; dt = 90, every 30 day
!   0           0           5760        1      24                ! every hr; dt = 150, every 10 day
!   0           0           2880        1      12                ! every hr; dt = 300, every 10 day
=====!

```

Figure 3-9. Card Image C4.

Card Image C4 (Figure 3-9) consists of restart options and a model simulation progress screen-show control options.

If you select **restart_out** = -1, then it indicates:

- you want to generate a restart file (**restart_***.out**) once at the end of the run; i.e., only one restart file will be generated at the last time step of the simulation (i.e., at **ndt** time step or $dt * ndt [s] = 150 * 17280 [s] = 30 [days]$). Thus, GOM will produce only one restart file, **restart_0017280.out**

If you select **restart_out** = 1 and **restart_freq** = 5760 (as shown in Figure 3-9), for example, and that indicates:

- you want to generate restart file (**restart_***.out**) every 5760 time steps; i.e., restart files will be generated every 10 days of simulation ($dt * restart_freq [s] = 150 * 1440 [s] = 10 [days]$). Thus, GOM will produce **restart_0005760.out**, **restart_0011520.out**, ...

Then, you can use one of the restart files as an initial condition setting **restart_in** = 1 (note that you should copy the restart file from **./output** to **./input** and change the file name to **restart.inp**) if you want to restart the model simulation from a certain time.

```

=====!
C5_01 Momentum equation variables I - Propagation/Nonlinear advection
! Note: The ELM method is used for the (1) nonlinear advection and (2) Coriolis terms
!
!   theta          [r]   Implicitness paramter for the propagation term. 0.5 <= theta <= 1.0
!   advection_flag  [i]   0/1   off/on, Turn off/on nonlinear advection: absolute criteria
!   adve_onoff_depth [r]   [m]   Turn off nonlinear advection below this depth: conditional criteria
!                               This is the additional off/on option for nonlinear advection
!   ELM_backtrace_flag [i] 0   Constant ELM subdivisions, [ELM_sub_iter], will be used.
!                               1   The number of subdivisions, [ELM_sub_iter], will be automatically calculated in the code
!                               based on the local velocity gradient.
!                               2~  Other integer numbers will make ELM unconditionally stable, but accuracy is not guaranteed.
!   ELM_sub_iter     [i]   Number of sub iteration for ELM
!                               If ELM_backtrace_flag is off (==0), [ELM_sub_iter] will be used.
!   ELM_min_iter     [i]   Number of minimum iteration for ELM. When ELM_backtrace_flag == 1
!   ELM_max_iter     [i]   Number of maximum iteration for ELM. When ELM_backtrace_flag == 1
!                               If ELM_backtrace_flag is on (==1), [ELM_sub_iter] will be calculated in the range:
!                               ELM_min_iter < ELM_sub_iter < ELM_max_iter
!
C5_01 theta      advection_flag  adv_onoff_depth  ELM_backtrace_flag  ELM_sub_iter  ELM_min_iter  ELM_max_iter
      0.6         1              0.01                0                  1              1              100
=====!
C5_02 Momentum equation variables II - Bottom friction
!   bf_flag         [i]   Bottom friction on/off option
!                               0 = No bottom friction
!                               1 = Chezy-Manning equation
!                               2 = Quadratic friction law (log law)
!   bf_varying      [i]   Spatially varying or constant bottom friction?
!                               0 = spatially constant bottom friction; the constant 'manning' or 'bf_height' will be used.
!                               1 = spatially varying bottom friction; bottom_roughness.inp is required.
!   manning         [r]   Manning's n (for Chezy-Manning equation),
!                               This value will be used if bf_flag == 1 & bf_varying == 0
!   bf_height       [r]   Bottom roughness height, z0 (for log law) in [m],
!                               This value will be used if bf_flag == 2 & bf_varying == 0
!   von_Karman      [r]   von Karman constant (for log law); typical value = 0.41
!
C5_02 bf_flag  bf_varying  manning  bf_height  von_Karman
      1         0          0.010   0.00001   0.41
=====!
C5_03 Momentum equation variables III - Baroclinic gradient/wetting & drying/Coriolis
!
!   baroclinic_flag [i] 0/1 off/on, baroclinic simulation
!                               0 Baroclinic density gradient effect will be ignored.
!                               1 Baroclinic density gradient effect will be included.
!   ana_density     [i] 0   Analytical density is off; the full equation of state will be used.
!                               1 Use a simple linear equation of state (rho = 1000.0 + 0.7*salinity)
!                               2 Use this option for test (user must specify own equation in calculate_analytical_density.f90).
!   ref_salt        [r] Reference salinity in [psu].
!                               If salinity transport is off (i.e., is_tran(1) == 0), this value will be used for the density calculat
!   ref_temp        [r] Reference water temperature in [Celcius].
!                               If temperature transport is off (i.e., is_tran(2) == 0), this value will be used for the density calcu
!   dry_depth       [r] [m] Should be greater than 0.0 [m] (0.01 m is recommended).
!                               If depth <= dry_depth, it will be a dry element.
!   Coriolis_option [i] 0 No Coriolis
!                               1 Constant Coriolis from given latitude in C1 (f-plane approximation)
!                               f = 2*omega*sin(mid_latitude)
!                               2 Variable Coriolis (by beta-plane approximation); [coordinate_system] in C1 must be 2 (i.e., lon/lat
!                               f = f0 + beta*y; where f0 = 2*omega*sin(mid_latidue); GOM will calculate f at each face center.
!
C5_03 baroclinic_flag  ana_density  ref_salt  ref_temp  dry_depth  Coriolis_option
      1                 0           30.0     20.0     0.01      1
=====!
C5_04 Momentum equation variables IV - Horizontal & Vertical diffusion
! In the case of momentum diffusion, they are called 'eddy viscosities'.
! For scalar fields such as temperature or salinity, the coefficients are called 'eddy diffusivities'. They are on the order of
!
!   Smagorinsky_parameter [r] Smagorinsky parameter in Smagorinsky-Lilly model.
!                               Usually it has the value: 0.0 ~ 0.25. See more details in 'solve_momentum_equation.f90'.
!   Ah_0                  [r] If this is set to 0.0, then the Smagorinsky-Lilly model will be turned off.
!                               Background horizontal eddy viscosity [m2/s]; for momentum diffusion.
!   Kh_0                  [r] This value will be added to the Smagorinsky model results.
!                               Background horizontal eddy diffusivity (Kh); for other diffusion.
!                               This value will be added to the Smagorinsky model results.
!                               Kh varies from near zero to order 10^4 [m2/s] (Cole et al, 2015); typically 10^2 ~ 10^4 [m2/s]
!   Av_0                  [r] Background vertical eddy viscosity [m2/s]; for momentum diffusion.
!                               typical value = 1.e-2 [m2/sec]
!   Kv_0                  [r] Background vertical eddy diffusivity [m2/s]; for other diffusion.
!                               typical value = 1.e-4 [m2/sec]
!
C5_04 Smagorinsky_parameter  Ah_0  Kh_0  Av_0  Kv_0
      0.021                  0.00  00.0  0.00  1.e-4
=====!

```

Figure 3-10. Card Image C5_01 ~ C5_04.

Card Images C5_01 through C5_04 are the momentum equation-related variables, and each card, unlike other sub-cards, is independent of each other as previously described. GOM adapted the semi-implicit scheme (so-called θ

scheme) for solving some parts of the governing equations, and the scheme is stable when the implicitness factor, θ , **theta** is in the range of $0.5 \leq \theta \leq 1.0$.

The **advection_flag** is an absolute criterion to turn off/on the nonlinear advection term while the following **adv_onoff_depth** is a conditional criterion. In this example, **adv_onoff_depth** = 0.01 indicates that the nonlinear advection term will be turned off if a cell depth falls below 0.01 meter.

You can either turn off or on **baroclinic_flag** while **transport_flag** in C7 is on. If **baroclinic_flag** is off while **transport_flag** is on, it means the transportive quantity (e.g., salinity or temperature) is treated as a tracer, and it doesn't affect the density flow. If, however, **baroclinic_flag** is on while **transport_flag** is on, it means the concentration of the salinity or temperature contributes to the water flow affecting water density. Thus, you can decide either barotropic or baroclinic simulation with this option.

GOM automatically checks whether an element is dry or wet comparing each element's depth with **dry_depth**. If the depth of an element is less than the given **dry_depth**, the element will be treated as a dry cell. Only elements greater than **dry_depth** will be treated as a wet element, and the momentum equation process will be conducted. Thus, this value must be greater than 0.0 m (the typical value is 0.01 m).

In GOM, a variable horizontal eddy viscosity (A_h) is calculated using the Smagorinsky subgrid-scale scheme (it is also known as the Smagorinsky-Lilly model) (Smagorinsky, 1963), which can be generally written in 2D Cartesian coordinates as (Ji, 2017):

$$A_h = C_s \Delta x \Delta y \left[\left(\frac{\partial u}{\partial x} \right)^2 + \left(\frac{\partial v}{\partial y} \right)^2 + \frac{1}{2} \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right)^2 \right]^{1/2} \quad (3-1)$$

where C_s is the Smagorinsky constant (and it is defined with the **Smagorinsky_parameter** in C5_04), Δx and Δy are the model grid sizes in x and y direction, respectively. The Smagorinsky constant has typical values between 0.1 and 0.2. Then, you can add additional values on top of the calculated eddy viscosity using the **Ah_0** in C5_04. Thus, the final eddy viscosity will be the sum of **Ah_0** and A_h which calculated from Equation (3-1).

The vertical turbulent closure model has not been implemented in the current version of the GOM. Thus, a user must provide the vertical eddy viscosity and diffusivity manually.

```

=====!
C6   Matrix solver option for Jacobi Preconditioned Conjugate Gradient (PCG) method:
!
!   max_iteration_pcg [i]      Maximum number of iteration
!   error_tolerance   [r]      The stopping criterion or approximate relative accuracy desired in the final computed solution.
!   pcg_result_show   [i]      0/1   off/on, PCG solver results show
!
!                               If it is on, GOM will show the converging iteration number both on the screen and the log file, run
C6   max_iteration_pcg   error_tolerance   pcg_result_show
      1000                5.e-6            0
=====!

```

Figure 3-11. Card Image C6.

Card Images C6 provides a matrix solver option. The Jacobi Preconditioned Conjugate Gradient (PCG) method is used to solve the sparse matrix system that arises when solving the free surface equation in GOM. You can set the **error_tolerance**, which affects both convergence speed and accuracy of the result.


```

=====!
C7   off/on tranport equations & selection of the solver
!
!   transport_flag   [i]   0/1   off/on, global turn off/on flag for transport equation
!   If transport_flag == 0, following variables will be automatically turned off.
!   transport_solver [i]   1/2   Transport equation solver selection option
!   1   TVD scheme
!   2   Eulerian Lagrangian Method (ELM)
!
C7   transport_flag   transport_solver
!   1   1
=====!
C7_1 TVD solver
!   Activate if transport_flag == 1 & transport_solver == 1 in C7
!
!   trans_sub_iter   [i]   N   Number of sub iteration (sub-cycling) for the TVD transport equation
!   0,1   No sub-iteration
!   2~N   Number of sub-iterations
!   h_flux_limiter   [i]   0~3   Flux limiter functions for horizontal:
!   0: No flux limiter (i.e., 1st-order upwind), 1: Minmod, 2: van Leer, 3: Superbee
!   v_flux_limiter   [i]   0~3   Flux limiter functions for vertical:
!   0: No flux limiter (i.e., 1st-order upwind), 1: Minmod, 2: van Leer, 3: Superbee
!
C7_1 trans_sub_iter   h_flux_limiter   v_flux_limiter
!   0   0   0
!   3   0   0
!   6   3   3
!   10  1   1
!   30  0   0
!   120 1   1
!   150 3   3
=====!
C7_2 Turn off/on transport variables
!   Activate if transport_flag == 1
!
!   is_tran [i] 0/1 off/on, transported scalar materials; note, currently GOM has two materials; it will be expanded step by
!   is_tran[1] = salinity
!   is_tran[2] = temperature
!   num_tran_ser [i] Total number of time series for each tranported material
!   If > 0, should have '***_ser.inp'
!   tran_ser_shape [i] There are two shapes of '***_ser.inp' depending on a user's preference
!   1 = ***_ser1.inp (useful if each station has different time record)
!   2 = ***_ser2.inp (useful if each station has identical time record)
!
C7_2 is_tran num_tran_ser tran_ser_shape
!   1 3 1 ! salinity; salt_ser.inp
!   1 3 1 ! temperature; temp_ser.inp
=====!
C7_3 Heat transport control variables 1
!   Activate if transport_flag == 1 & is_tran[2] == 1; i.e., only if temperature transport is activated.
!
!   heat_option [i] 1/2 Short-wave solar radiation computation option
!   1 = Term-by-Term calculation
!   2 = Equilibrium temperature (not yet implemented)
!   sol_swr [i] 1/2 option to calculate short-wave solar radiation
!   1 = short-wave solar radiation will be calculated from model domain's lon/lat & time info
!   2 = short-wave solar radiation will be read in from air_ser.inp
!   fWz_a [r] empirical evaporation coefficient a; default = 9.2
!   fWz_b [r] empirical evaporation coefficient b; default = 0.46
!   fWz_c [r] empirical evaporation coefficient c; default = 2.0
!   wind_height [r] wind speed measurement height [m] in air_ser.inp; typically either 2.0[m] or 10.0[m]
!
C7_3 heat_option sol_swr fWz_a fWz_b fWz_c wind_height
!   1 2 9.2 0.46 2.0 2.0
=====!
C7_4 Heat transport control variables 2 - surface/bottom boundary related coefficients
!   Activate if transport_flag == 1 & is_tran[2] == 1; i.e., only if temperature transport is activated.
!
!   light_extinction [r] light extinction coefficient, Kd, [1/m]; default = 0.45
!   sol_absorb [r] fraction short-wave absorbed on water surface; default = 0.45
!   sed_water_exchange [r] coefficient of sediment/water heat exchange, [W/m2/C]; default = 0.3
!   T_sed [r] sediment temperature [degree C]; typical value = 11.5 C
!   sed_temp_coeff [r] sediment temperature coefficient (0 ~ 1); Heat lost to sediments that is added back to water column
!   e.g., 0: 0% of the shortwave solar radiation is reradiated into the water column resulting in a loss
!
C7_4 light_extinction sol_absorb sed_water_exchange T_sed sed_temp_coeff
!   0.45 0.45 0.3 11.5 0.0
=====!

```

Figure 3-12. Card Image C7.

Card Images C7 family (C7 through C7_4) are related to the transport model. You can either turn off or on the transport equation using the **transport_flag**. GOM provides two transport equation solving algorithms: (1) A high-resolution transport method with the 2nd-order semi-implicit Total Variation Diminishing (TVD) scheme with flux

limiter functions, and (2) Eulerian Lagrangian Method (ELM); you can select each algorithm with **transport_solver**. When you select the TVD scheme, you can choose one of the four transport models: (0) 1st-order upwind method, (1) Minmod limiter, (2) van Leer limiter, and (3) Superbee limiter. The stability criterion of the TVD transport model is (Casulli and Zanolli, 2005):

$$\Delta t \leq \frac{P_i \Delta z_{i,k}^n}{2 \sum_{j \in S_i^+} |Q_{j,k}^{n+\theta}| + 2 |Q_{i,k+1/2}^{n+\theta}| + \sum_{j \in S_i^+ \cup S_i^-} d_{j,k}^{n+\theta}} \quad (3-2)$$

However, sometimes this criterion is too severe comparing to the hydrodynamic model and this transport model part becomes a bottleneck. In this case, GOM allows you to use the sub-cycling (or sub-iteration) approach by setting **trans_sub_iter** ≥ 2 . If sub-cycling is used, the transport model will have a smaller time step than the hydrodynamic model (i.e., $dt_{transport} = dt/trans_sub_iter$), and GOM will become stable. Note that 0 or 1 for **trans_sub_iter** have the same effect since 0 means no sub-iteration, and 1 means 1 sub-iteration.

If **transport_flag** is on, you must turn on C7_1, otherwise, turn off C7_1 either by deleting the line or putting “!” at the beginning of the line. Note that if ELM is selected (i.e., **transport_solver** = 2), C7_1 must be turned off since C7_1 only works with the TVD scheme.

The current version of the GOM only has Salinity and Temperature transport models as shown in C7_2. Each tranportive material may have several time series depending on how many boundary conditions are required in the model domain, and this can be set by **num_tran_ser**. If **num_tran_ser** > 0 for both salinity and temperature, **salt_ser.inp** and **temp_ser.inp** should exist in *.input* folder

You can use different time series input file formats: (1) *****_ser1.inp** and (2) *****_ser2.inp**. If *****_ser1.inp** option is selected, the time series at each station has different time record, thus the time series data at each station will be stacked vertically. If *****_ser2.inp** option is selected, the time series at each station has identical time record, thus the time series at each station will be stacked horizontally at each time.

Heat (temperature) transport control variables are stored in C7_3 and C7_4, and these cards should be activated only if temperature transport is activated at C7_2 with **is_tran[2]** = 2. There are two approaches when solving the short-wave solar radiation: (1) Term-by-Term and (2) Equilibrium temperature. However, only the Term-by-Term calculation option is implemented in the current version.

```

=====!
C8   Model termination control
!
!   terminate_check_freq:  [i]   N   Check model termination criteria at every terminate_freq period (N * dt seconds)
!   eta_min_terminate:     [r]   Model termination criterion by minimum water surface elevation [m]
!   eta_max_terminate:     [r]   Model termination criterion by maximum water surface elevation [m]
!   uv_terminate:          [r]   Model termination criterion by maximum horizontal velocities [m/s]
!   salt_terminate:        [r]   Model termination criterion by maximum salinity [ppt]
!   temp_terminate:        [r]   Model termination criterion by maximum water temperature [Celcius]
!
C8   terminate_check_freq  eta_min_terminate  eta_max_terminate  uv_terminate  salt_terminate  temp_terminate
!   2880                   -3.0              3.0              5.0           50.0           50.0 ! every day; dt = 30
!   1440                   -3.0              3.0              5.0           50.0           50.0 ! every day; dt = 60
!   960                    -3.0              3.0              5.0           50.0           50.0 ! every day; dt = 90
!   576                    -3.0              3.0              5.0           50.0           50.0 ! every day; dt = 150
!   288                    -3.0              3.0              5.0           50.0           50.0 ! every day; dt = 300
!   9999999                -3.0              3.0              5.0           50.0           50.0 ! skip checking
!
=====!
! The end of the first set.
! C20 ~ C29, Tidal & River Boundary Conditions
!
=====!

```

Figure 3-13. Card Image C8.

Card Image C8 consists of model termination control variables. Sometimes, the model may produce unrealistic results, but it keeps running without termination. Therefore you may realize (or find) the faulty simulation at the end of the simulation. To avoid this effort, GOM provides these options. Using this card information, you can easily detect possible errors and terminate the model simulation.

Card Image C8 is the end of the first set, and the second set starts from C20.


```

=====
! The end of the first set.                                     !
! C20 ~ C29, Tidal & River Boundary Conditions                 !
=====
C20  Open boundary (tidal eta) information
!
!   num_ob_cell:      [i]  Total number of open boundary cells (elements) - for eta only
!                        [num_ob_cell] = [num_tide_bc] + [num_eta_bc]
!                        If > 0, either activate C20_1 or provide open boundary information in Obc_info.inp
!                        If == 0, following variables will be ignored.
!   ob_info_option:   [i]  How do you want to provide the open boundary (eta) information?
!                        0 In the following sub-card; following sub-card must activated
!                        1 Provide the open boundary (eta) information in the additional ob_info.inp;
!                          if this is selected, do not activate the following sub-card.
!   num_tide_bc:      [i]  Number of tidal boundary (periodic forcing using tidal constituents)
!   num_harmonic_ser: [i]  Total number of harmonic series
!                        If > 0, should have 'harmonic_ser.inp'
!   check_tide:       [i]  1 = Write harmonic tide checking file: check_tide.dat
!                        Harmonic tide at each station: every 1 hour.
!                        Not yet activated
!   num_eta_bc:       [i]  Number of surface elevation boundary (eta given, general purpose)
!   num_eta_ser:      [i]  Total number of eta time series
!                        If > 0, should have 'eta_ser.inp'
!   check_eta:        [i]  1 = Write eta series checking file: check_eta.dat
!                        Interpolation result: default = every 30 minutes.
!                        Not yet activated
!   eta_ser_shape:    [i]  There are two shapes of 'eta_ser.inp' depending on a user's preference
!                        1 = eta_ser1.inp (useful if each station has different time record)
!                        2 = eta_ser2.inp (useful if each station has identical time record)
C20
C20  num_ob_cell      ob_info_option
C20  num_tide_bc      num_harmonic_ser      check_tide
C20  num_eta_bc       num_eta_ser           check_eta      eta_ser_shape
75
0
75
0
2
0
2
! eta_ser1.inp is also provided for the reference
=====
C20_1 Eta boundary information
!
!   Activate if num_ob_cell > 0 .and. ob_info_option == 0.
!   Should have 'num_ob_cell' information (lines).
!
!   serial_num      [i]  Station number (dummy number)
!   ob_nodes(i,1)   [i]  First node of the station (or the element, in counterclockwise direction)
!   ob_nodes(i,2)   [i]  Second node of the station (or the element, in counterclockwise direction)
!   ob_eta_type     [i]  Open boundary surface elevation type:
!                        -1 Radiation bc
!                          No input is required; elevations will be computed as average of surrounding elevations.
!                        1 This boundary is forced by tidal harmonic constituents; harmonic_ser.inp is required; default = cos
!                        2 This boundary is forced by time history of elevation; eta_ser.inp is required
!   harmonic_ser_id [i]  ID number of harmonic series at this station.
!                        If ob_eta_type == 1, this should be greater than 0.
!   eta_ser_id      [i]  ID number of surface elevation time series at this station.
!                        If ob_eta_type == 2, this should be greater than 0.
!   salt_ser_id     [i]  ID number of salinity time series at this station.
!                        If transport_flag > 0 & is_tran(1) > 0, this should be greater than 0
!   temp_ser_id     [i]  ID number of temperature time series at this station.
!                        If transport_flag > 0 & is_tran(2) > 0, this should be greater than 0
!   ob_name         [c]  dummy variable. This will not be read.
C20_1 serial_num ob_nodes(i,1) ob_nodes(i,2) ob_eta_type(i) harmonic_ser_id(:) eta_ser_id(:) salt_ser_id temp_ser_id ob
1 18718 18716 2 0 1 1 1
2 18716 18713 2 0 1 1 1
.
.
74 12402 12255 2 0 2 2 2
75 12255 12102 2 0 2 2 2
=====

```

Figure 3-14. Card Images C20 & C20_1.

Notice that the second card sets start from C20 not from C9 as mentioned previously. Through Card Images C20 and C20_1 you specify tidal open boundaries. You can set the tidal open boundary either by (1) using harmonic tidal constituents (setting **num_tide_bc** > 0, this requires *harmonic_ser.inp*) or by (2) providing measured water

surface elevation data (setting **num_eta_bc** > 0, this requires *eta_ser.inp*). Thus, the total tidal open boundary elements (**num_ob_cell**) must equal the sum of **num_tide_bc** and **num_eta_bc**.

When providing each open boundary element's node information in C20_1, the first (**ob_nodes(i,1)**) and the second (**ob_nodes(i,2)**) nodes must be provided in the counterclockwise order in an element, otherwise, GOM will not be able to find the corresponding element. Each open boundary element can have boundary salinity and temperature, and they are provided by setting **salt_ser_id** and **temp_ser_id**. There is one more useful option **ob_info_option**. When **ob_info_option** is activated, you can provide the following sub_card C20_1 in a separate input file *Obc_info.inp*, which has the exact form of C20_1. This option is quite useful when the open boundary consists of many elements; similarly, there is **Qbc_info_option** in C22.

```

=====!
C21  Setup spinup function for tide
!
!   tide_spinup           [i]   0/1   off/on, spinup for tide
!   tide_spinup_period    [r]   [day]
!   baroclinic_spinup     [i]   0/1   off/on, spinup for baroclinic
!   baroclinic_spinup_period [r] [day]
!
C21  tide_spinup           tide_spinup_period
C21  baroclinic_spinup     baroclinic_spinup_period
      0                    0.5                ! tide_spinup
      0                    0.5                ! baroclinic_spinup
=====!

```

Figure 3-15. Card Image C21.

Sometimes your model may crash at the beginning due to the strong barotropic or baroclinic gradient at the boundary elements if the boundary values and the initial values are quite different. In this case, you may need to activate this option to smooth the strong gradient.

```

=====!
C22  Include river boundary
!
!   num_Qb_cell:      [i]   Number of river or point source boundaries (Not node number but cell number) (Q given)
!                       If > 0, should activate the following sub-card & need 'q_ser.inp'
!   Qbc_info_option:  [i]   How do you want to provide the river boundary information?
!                       0 In the following sub-card; following sub-card must activated
!                       1 Provide the river boundary information in the additional Qbc_info.inp;
!                       if this is selected, do not activate the following sub-card.
!   num_Q_ser:        [i]   Total number of Q time series
!   check_Q:          [i]   1 = Write Q series checking file: check_Q.dat
!                       Interpolation result: default = every 30 minutes.
!   Q_ser_shape:      [i]   There are two shapes of 'q_ser.inp' depending on a user's preference
!                       1 = q_ser1.inp (useful if each station has different time record)
!                       2 = q_ser2.inp (useful if each station has identical time record)
!
C22  num_Qb_cell Qbc_info_option  num_Q_ser  check_Q  Q_ser_shape
      1          0                1         0        1
=====!
C22_1 River or point source boundary (Q given) information
!   Activate if num_Q_bc > 0. num_Q_bc lines are required.
!
!   serial_num:      [i]   Station number (dummy number)
!   Q_nodes(i,1):    [i]   First node of the station (or the element, in counterclockwise direction)
!   Q_nodes(i,2):    [i]   Second node of the station (or the element, in counterclockwise direction)
!   Q_ser_id:        [i]   ID number of river (Q) time series
!   Q_portion:       [r]   Portion of Q for Q_ser_id
!   Q_salt_ser_id    [i]   ID number of salinity time series at this station.
!   Q_temp_ser_id    [i]   ID number of temperature time series at this station.
!   Q_name:          [c]   River or point source ID name (dummy variable)
!
C22_1 serial_num  Q_nodes(i,1)  Q_nodes(i,2)  Q_ser_id(:)  Q_portion  Q_salt_ser_id  Q_temp_ser_id  !Q_name
      1          151          141           1         1.0        3             3             ! Mobile River (cell# = 145)
=====!

```

Figure 3-16. Card Image C22 & C22_1.

You can define river discharge boundary in C22 and C22_1. As in C20_1, when providing river boundary element's node information, the first (**Q_node(i,1)**) and the second (**Q_node(i,2)**) nodes must be provided in the counterclockwise order in an element. If a river boundary consists of several elements, you can set **Q_portion** to set each element's discharge portion to the given total discharge amount. This option is also useful if you want to test with reduced or increased river discharge without changing the river discharge data. If the river boundary condition includes salinity and temperature, you should point correct time series ID also.

```

=====!
C23   Include Withdraw/Return (WR) point boundary
!     This option differs from river boundary; this option does not work with ELM transport
!
!     num_WR_cell:      [i]   Number of Withdraw/Return point source boundaries (Not node number but cell number) (Q given)
!                          If > 0, should activate the following sub-card
!
C23   num_WR_cell
      0
=====!
C23_1 Withdraw/Return (WR) point boundary information
!     Activate if num_WR_cell > 0. num_WR_cell lines are required.
!
!     serial_num      [i]   Station number (dummy number)
!     WR_nodes(i,1)   [i]   First node of the station (or the element, in counterclockwise direction)
!     WR_nodes(i,2)   [i]   Second node of the station (or the element, in counterclockwise direction)
!     WR_layer:       [i]   Withdraw/Return vertical layer
!                          This is the main difference between river boundary and Withdraw/Return boundary
!                          999 = anytime surface layer
!                          1 ~ N = specific vertical layer
!                          0 = anytime bottom layer
!     WR_Q_ser_id     [i]   ID number of Withdra/Return (Q) time series in q_ser.inp
!     WR_portion      [r]   Portion of Q for WR_Q_ser_id
!     WR_salt_ser_id   [i]   ID number of salinity time series at this station.
!     WR_temp_ser_id   [i]   ID number of temperature time series at this station.
!     WR_name:        [c]   Withdraw/Return point source ID name (dummy variable)
!
C23_1 serial_num WR_nodes(i,1) WR_nodes(i,2) WR_layer WR_Q_ser_id WR_portion WR_salt_ser_id WR_temp_ser_id ! WR_name
!     1          2121          2123          999      3          50.00      4          4          ! test
=====!

```

Figure 3-17. Card Image C23 & C23_1.

The C23 controls withdraw and return flows. This option differs from the river boundary condition in terms of the vertical location. With this option, you can select specific vertical layers of withdraw and return flows with **WR_layer**. When you use withdrawal flow, you need to specify salt_ser_id with 0.0 salinity if you are simulating transport (i.e., you have to have 0.0 salinity timeseries in *salt_ser.inp*).

```

=====!
C24 Include Source/Sink (SS) point boundary
! This option differs from river boundary & withdraw/return boundary; this option does not work with ELM transport
!
! num_SS_cell: [i] Number of Source/Sink point source boundaries (Not node number but cell number)
! If > 0, should activate the following sub-card
C24 num_SS_cell
0
=====!
C24_1 Source/Sink (SS) point boundary information
! Activate if num_SS_cell > 0. num_SS_cell lines are required.
!
! serial_num [i] Station number (dummy number)
! SS_cell [i] Cell (or element) number of the station
! SS_layer [i] Source/Sink vertical layer
! 999 = anytime surface layer
! 1 ~ N = specific vertical layer
! 0 = anytime bottom layer
! SS_Q_ser_id [i] ID number of Source/Sink (Q) time series
! SS_portion [r] Portion of Q for SS_Q_ser_id
! SS_salt_ser_id [i] ID number of salinity time series at this station.
! SS_temp_ser_id [i] ID number of temperature time series at this station.
! SS_name: [c] Source/Sink point source ID name (dummy variable)
!
C24_1 serial_num SS_cell SS_layer SS_Q_ser_id SS_portion SS_salt_ser_id SS_temp_ser_id ! SS_name
! 1 3662 999 3 10.00 4 4 ! test
=====!

```

This option differs from the river boundary and withdraw/return boundary condition in terms of the horizontal and vertical locations. This boundary can be anywhere in the model domain, i.e., even inside the water body. However, this option does not work with ELM transport, thus you should select TVD transport option if you activate this option. Note that the water volume will be added or extracted from the cell's surface not from the designated layer, but the transport process will be activated at the designated layer.


```

=====!
C25  Include earth equilibrium tidal potential (or Solid Earth Tide)
!
!   no_Etide_species      [i]   Number of Earth equilibrium tidal potential species
!   [no_Etide_species] should be less than [max_no_tidal_constituent]
!
!   Etide_cutoff_depth    [r]   [m], if the water depth is less than this, earth tide will not be added.
!   See more details in [solve_momentum_equation.f90]
!
C25  no_Etide_species  Etide_cutoff_depth
      0                40.0
=====!
C25_1 Activate if [no_Etide_species] > 0
!
!   [no_Etide_species] lines are required
!
!   Etide_species         [i]   1,2,3,...
!   Etide_amplitude        [r]   [m]
!   Etide_frequency        [r]   [0,1,2,...,n times]/day, or cycle per day (cpd)
!   Etide_nodal_factor     [r]   see more details in any reference
!   Etide_astro_arg_degree [r]   [degree]
!
C25_1 Etide_species(i)  Etide_amplitude(i)  Etide_frequency(i)  Etide_nodal_factor(i)  Etide_astro_arg_degree(i)
!      0                0                0                0                0
=====!
! C25_1 is the end of the second set.
! C30 ~ C39, Air/Sea Boundary Conditions
=====!

```

You can include the earth equilibrium tidal potential (or Solid Earth Tide) here.

The C25_1 is the end of the second card set.

```

=====
! C25_1 is the end of the second set.
! C30 ~ C39, Air/Sea Boundary Conditions
!=====
C30 Wind & pressure data type #1: using simple measured data from wind station
! General wind information from wind stations (not storm type winds)
!
! wind_flag [i] 0 There is no wind station data.
! 1 Wind information in windp_ser.inp will be used.
! 2 Analytical wind stress will be used in [ana_windp.f90].
! airp_flag [i] 0 There is no air pressure station data.
! 1 Air pressure information in windp_ser.inp will be used.
! num_windp_ser [i] Total number of wind and pressure series (stations) in windp_ser.inp
! If one of wind_flag or pressure_flag == 1, this variable will be activated.
! Currently, maximum 3 windp stations are allowed.
! If more than 3 stations, consider to use wind model data as in C32.
! wind_formular [i] Select one of the following wind stress formulations:
! Default formular: Smith's (1980) equation (when you choose other than 1 ~ 3)
! 1 = Garratt's (1977) formula: This method is also used in CH3D.
! 2 = Smith's (1980) formula: This method is the default in GOM.
! 3 = Wu's (1982) formula: This method is also used in SWAN.
! wind_spinup [i] 0/1 off/on, Spinup for wind
! wind_spinup_period [r] Wind spinup period in [day]
!
C30 wind_flag airp_flag num_windp_ser wind_formular wind_spinup wind_spinup_period
0 0 0 2 0 0.0
=====

```

Figure 3-18. Card Image C30.

Card Image C30 through C32 specify Air/Sea boundary conditions. You can use three different approaches to include atmospheric (mostly wind) effects: (1) using wind station data, (2) using hurricane type data, and (3) using analytical hurricane data. The first one, i.e., C30, is the most basic option, and you can use this when you want to use wind and air pressure station data directly. If this option is selected, GOM will interpolate atmospheric station data into a model grid nodal point using the Inverse Distance Weighting (IDW) interpolation method.


```

=====!
C31  Wind & pressure data type #2: using measured hurricane data or wind model data
!    Use hurricane data which includes wind and pressure
!    Data from National Hurricane Center (NOAA/NHC) or Hurricane Research Division (NOAA/HRD)
!
!    hurricane_flag      [i]    0  Wind and pressure (Hurricane type) will not be calculated,
!                               1  Wind and pressure will be calculated, hurricane (wind and pressure) data file hurricane_ser.i
!                               Following variables will be activated.
!    hurricane_data_type [i]    1  Wind and pressure data should be ascii format
!                               2  wind and pressure data should be binary format
!    hurricane_interp_method [i] 1  time-only interpolate; use this when you only have wind model data
!                               2  time & space interpolate: Inverse Distance Weighting (IDW) Interpolation
!                               Hurricane center data file, hurricane_center.inp, should exist
!    hurricane_dt        [i]    Hurricane data interval in [sec]
!                               Number of hurricane data set in hurricane_ser.inp should be greater than:
!                               [(hurricane_end_jday - hurricane_start_jday)*(86400/hurricane_dt) + 1
!                               188 ~ 193 julian day
C31
C31  hurricane_flag      hurricane_data_type      hurricane_interp_method      hurricane_dt
      1                  1                      1                      21600      ! 6 hourly data
=====!
C31_1 Hurricane start/end-year/month/day
!    Activate if hurricane_flag == 1
!
!    hurricane_start_year [i]    Hurricane (or wind model data) start year
!    hurricane_start_month [i]   Hurricane (or wind model data) start month
!    hurricane_start_day [i]     Hurricane (or wind model data) start day
!    hurricane_end_year [i]      Hurricane (or wind model data) end year
!    hurricane_end_month [i]     Hurricane (or wind model data) end month
!    hurricane_end_day [i]       Hurricane (or wind model data) end day
!
C31_1 hurricane_start_year      hurricane_start_month      hurricane_start_day
C31_1 hurricane_end_year       hurricane_end_month       hurricane_end_day
      2010                      8                          1
      2010                      9                          30
=====!

```

Figure 3-19. Card Image C31 and C31_1.

If you want to use either wind model data or want to conduct hurricane simulation, the second option, i.e., C31, can be used. The main conceptual difference of the data set between C30 and C31 is whether the data is provided at entire model grid nodes (C31, I defined this data set as a hurricane type) or not (C30). For example, a wind model data covers the entire model grid, thus you must choose the second data type. Then, GOM distinguishes the provided data by either wind model data or hurricane data. Thus, you must select **hurricane_interp_method** = 1 for the general wind model. However, for the hurricane simulation, it is better to select **hurricane_interp_method** = 2 to use time and space interpolation. The main difference is that hurricane simulation requires *hurricane_center.inp* which is the hurricane center tracking (hurricane eye's x and y location at each time step) data. You can also test **hurricane_interp_method** = 1 for the hurricane simulation, but this option will give less accurate results than **hurricane_interp_method** = 2. When **hurricane_inter_method** = 2 is selected, the results will be better and accurate, but the simulation time will be increased by doing both time and space interpolation. When choosing wind model or hurricane data type, wind and pressure data at entire nodes must be provided with *hurricane_ser.inp*. The *hurricane_ser.inp* is independent from the data start time which is defined in C3 which provides other data's start and end time. There is a reason why I used this approach. The *hurricane_ser.inp* becomes quite big when the model grid is big, and that is why I selected this approach to reduce this file size.

```

=====
C32  Wind & pressure data type #3: using analytical hurricane data
!   (not yet finished)
!   Later, I will also include Willoughby et al. (2006)'s model
!   Use Holland storm surge model
!
!   holland_flag      [i]   0/1   off/on, Following variables will (or not) be activated.
!   holland_start_jday [r]
!   holland_end_jday  [r]
!
C32  holland_flag      holland_start_jday  holland_end_jday
      0                0.0                0.0
!=====
! C32 is the end of the third set.
! C40 ~ C49: Output Control Options
!=====

```

Figure 3-20. Card Image C32.

The 3rd atmospheric data type is the analytical hurricane data type as defined at C32. You can generate analytical wind and pressure data using Holland's storm surge model (Holland, 1980).

The card image C32 is the end of the third set.

```

=====!
! C32 is the end of the third set.                                     !
! C40 ~ C49: Output Control Options                                   !
=====!
C40   Check grid
!
!   check_grid_2D           [i]   0/1   off/on, write a 2D grid checking file - origin shifted version (for the nice view)
!   check_grid_2DC          [i]   0/1   off/on, write a 2D grid checking file - original version
!   check_grid_3D           [i]   0/1   off/on, write a 3D grid checking file
!   check_grid_format        [i]   1/2/3 Tecplot/VTK/both file format
!   check_grid_info          [i]   0/1   off/on, write additional grid information - not yet activated
!   check_grid_unit_conv     [r]           unit conversion factor for the coordinate
!                                     1.0 = [m], original unit in the grid information
!                                     0.001 = [km], change the unit to kilometer
!
C40   check_grid_2D      check_grid_2DC      check_grid_3D      check_grid_format check_grid_info      check_grid_unit_conv
      1                  1                  1                  1              0              0.001
=====!

```

Figure 3-21. Card Image C40.

Card Image C40 through the end covers output control options. The GOM provides two types of direct output data formats: (1) Tecplot, and (2) VTK file format. Tecplot is a commercial product, and it is easy to use. The Visualization Toolkit (VTK) can be used for several open visualization tools such as ParaView and VisIt. With this approach, you can easily check your output results. Note that Tecplot option will give you faster simulation (file writing) speed than VTK file option. Thus, if you have Tecplot license, I strongly suggest you to use Tecplot option rather than VTK option.

Throughout these output control options, you may easily understand what you need to select with the description in each card image, thus I will skip additional explanations.

```

=====!
C41
!   tser_station_num   [i]   Total station for time series output files
!   tser_info_option   [i]   How do you want to provide the time series station information?
!                               0   In the following sub-card; following sub-card must activated
!                               1   Provide the time series station information in the additional tser_station_info.inp;
!                               if this is selected, do not activate the following sub-card.
!   tser_hloc          [i]   Time series output horizontal location
!                               1 = cell center
!                               2 = node
!   tser_frequency     [i]   Time series output file (***_tser.dat) frequency
!   tser_time           [i]   output file time unit
!                               1 = second, 2 = minute, 3 = hour, 4 = day
!   tser_format         [i]   1/2/3 Tecplot/VTR/both file format
!
C41  tser_station_num  tser_info_option  tser_hloc  tser_time  tser_frequency  tser_format
!   5                  0                1          4          60             1             ! every hr; dt = 60s
!   5                  0                1          4          40             1             ! every hr; dt = 90s
!   5                  0                1          4          24             1             ! every hr; dt = 150s
!   5                  0                1          4          12             1             ! every hr; dt = 300s
!   5                  0                1          4           6             1             ! every hr; dt = 600s
!   5                  0                1          4           2             1             ! every hr; dt = 1800s
=====!
C41_1 Read individual time series station element number
!   Activate if tser_station_num > 0
!
!   serial_num         [i]   Station number (dummy number)
!   tser_station_cell   [i]   Station element number: will be used only if [tser_loc == 1]
!   tser_station_node   [i]   Station node number: will be used only if [tser_loc == 2]
!   tser_station_name   [c]   Station name. Maximum 25 characters.
!
C41_1 serial_num  tser_station_cell  tser_station_node  tser_station_name
!   1             772                999                'MSD'
!   2             22197              999                'DPI'
!   3             14200              999                'MBL'
!   4             6975               999                'BOS'
!   5             1949               999                'MEP'
=====!
C41_2 Activate/deactivate time series variables
!   Activate if tser_station_num > 0
!
!   tser_eta           [i]   0/1   off/on, write water surface elevation time series: eta_tser_from_msl.xxx,
!   tser_H              [i]   0/1   off/on, write total water depth: H_tser.xxx
!   tser_u              [i]   0/1   off/on, write vertically averaged velocity time series: u_tser.xxx
!   tser_v              [i]   0/1   off/on, write vertically averaged velocity time series: v_tser.xxx
!   tser_salt           [i]   0/1   off/on, write vertically averaged salinity time series: salt_tser.xxx
!   tser_temp           [i]   0/1   off/on, write vertically averaged temperature time series: temp_tser.xxx, not yet included
!   tser_pressure       [i]   0/1   off/on, write air pressure time series at surface: airp_tser.xxx, not yet included
!
C41_2 tser_eta  tser_H  tser_u  tser_v  tser_salt  tser_temp  tser_pressure
!   1           0       1       1       1          1          0
=====!

```

Figure 3-22. Card Image C41, C41_1, & C41_2.

If you have many time series output stations thus you do not want to keep a long list of them in this *main.inp*, you can select **tser_info_option** = 1 then provide the information in a separate input file, *tser_station_info.inp*. GOM uses the staggered grid system, thus velocities are calculated at element faces but most other variables (e.g., η , salinity, temperature) are calculated at the element center. However, here you can select the output location option either at a cell center or each nodal point with **tser_hloc**.

```

=====!
C42  2D output file (tec2D_###.dat or vtk2D_###.dat) control
!
!   IS2D_switch: [i]  0/1  off/on option
!   IS2D_flood_map [i] 0/1  off/on 2D maximum flood map; write one time at the end of simulation.
!   Works only when IS2D_switch is on
!   IS2D_format: [i]  1/2/3 Tecplot/VTK/both file format
!   IS2D_binary: [i]  0/1  Select file output format: ASCII or Binary
!   0 = ASCII
!   1 = Binary ! not yet activated
!   IS2D_File_freq:[i] 2D out file generation frequency:
!   A new file generation every [dt * IS2D_File_freq] times
!   This option only works with Tecplot files
!   IS2D_time: [i] output file time unit
!   1 = second, 2 = minute, 3 = hour, 4 = day
!   IS2D_frequency:[i] 2D contour plot frequency: display time = dt*IS2D_frequency [sec]
!   Note: IS2D_File_frequency does not affect on VTK files.
!   IS2D_start: [i] 2D data start time (simulation step)
!   IS2D_end: [i] 2D data end time (simulation step)
!   IS2D_unit_conv:[r] unit conversion factor for the coordinate
!   1.0 = [m], original unit in the grid information
!   0.001 = [km], change the unit to kilometer
!
C42  IS2D_switch  IS2D_flood_map
C42  IS2D_format  IS2D_binary  IS2D_File_freq
C42  IS2D_time    IS2D_frequency IS2D_start  IS2D_end  IS2D_unit_conv
!   1           0
!   1           0           14400
!   4           60           0           999999  0.001
!   1           0
!   1           0           9600
!   4           40           0           999999  0.001
!   1           0
!   1           0           5760
!   4           24           0           999999  0.001
!   1           0
!   1           0           2880
!   4           12           1           999999  0.001
!   1           0
!   1           0           1440
!   4           6           1           999999  0.001
!   ! dt = 60
!   ! every 10 day simulation
!   ! every 1 hr, from 55day
!   ! dt = 90
!   ! every 10 day simulation
!   ! every 1 hr; from 55day
!   ! dt = 150
!   ! every 10 day simulation
!   ! every 1 hr; from 55day (=31680)
!   ! dt = 300
!   ! every 10 day simulation
!   ! every 1 hr
!   ! dt = 600
!   ! every 10 day simulation
!   ! every 1 hr
!-----!
C42_1 2D output file (tec2D_###.dat or vtk2D_###.dat) variable selection
!
! It will be used only if (IS2D_switch == 1), so do no deactivate this card.
! Note: vertically averaged values will be printed out.
!
!   variable: [i] dummy variable id (do not change order)
!   IS2D_variable: [i] 0/1 off/on option for each variable
!
C42_1 variable  IS2D_variable(:)
eta           1
u             1
v             1
salt          1
temp          1
rho           0
=====!

```

Figure 3-23. Card Image C42 & C42_1.

With C42 and C42_1, you can control two-dimensional (2D) output file. If this option is activated, vertically averaged values will be written in a 2D output file format. You can also activate **IS2D_flood_map** to see the maximum flood map, which provides a useful information when you are working on coastal inundation projects.

The variables named with *****_File_freq** (e.g., **IS2D_File_freq**) control a file separation into pieces of sub-files. One-dimensional time series files are usually not that big, thus the file does not need to be separated. However, 2D or 3D output files are easily over several Gigabytes (GB), depending on the total number of elements. In this case, you can tell GOM how often generates sub-files, and a new file will be generated every, for example, $[dt \times IS2D_File_freq]$ second, and a new file will have **IS2D_File_freq** time data. For example, if **dt** = 150 sec and **IS2D_File_freq** = 5760 (as shown in C42), a new 2D output file will be generated every 10 days ($= 150 \text{ sec} \times 5760 = 864000 \text{ sec} = 10 \text{ days}$). A new file, e.g., *tec2d_****.dat*, where * denotes the file number: 1, 2, 3, ..., N, will be

generated every 10 days, and it will have hourly data sets (i.e., 240 data sets) if **IS2D_frequency** = 24 (i.e., $\text{dt} \times \text{IS2D_frequency} = 150 \text{ sec} \times 24 = 3600 \text{ sec}$). Note that only the first file will have one more data set since it includes the initial condition. Since the Tecplot file formats are lighter than VTK file formats, the simulation speed will be faster with the Tecplot file format options than the VTK file format options as mentioned previously.

You can also select output time period with **IS2D_start** and **IS2D_end**, i.e., you don't need to print out all simulation time period data.

Note that you should not deactivate (i.e., comment out) C42_1 even though **IS2D_switch** = 0; i.e., always keep C42_1 uncommented.


```

=====!
C43  3D output file (tec3D_###.out or vtk3D_###.dat) control
!    Currently, only for the surface elevation (eta)
!
!    IS3D_full_switch: [i]  0/1  off/on option for 'tec3D_full_***.dat' or 'vtk3D_full_***.dat'
!    IS3D_grid_format: [i]  1/2  3D grid view style; this will be used only if tecplot file format is selected.
!    1 = sigma-grid view; this is faster and lighter, but this will show modified view.
!    2 = z-grid view; this is slower and heavier, but this will show exact view.
!
!    IS3D_surf_switch: [i]  0/1  off/on option for 'tec3D_surf_***.dat' or 'vtk3D_surf_***.dat'
!    IS3D_format:      [i]  1/2/3 Tecplot/VTK/both file format
!    IS3D_binary:      [i]  0/1  Select file output format: ASCII or Binary
!    0 = ASCII
!    1 = Binary ! not yet activated
!
!    IS3D_File_freq:   [i]  3D out file generation frequency
!    A new file generation every [dt * IS2D_File_freq] times
!
!    IS3D_time:        [i]  output file time unit
!    1 = second, 2 = minute, 3 = hour, 4 = day
!
!    IS3D_frequency:   [i]  3D contour plot frequency: display time = dt*IS3D_frequency [sec]
!    Note: IS3D_File_freq does not affect on VTK files.
!
!    IS3D_start:        [i]  3D data start time (simulation step)
!    IS3D_end:          [i]  3D data end time (simulation step)
!    IS3D_unit_conv:    [r]  unit conversion factor for the coordinate
!    1.0  = [m], original unit in the grid information
!    0.001 = [km], change the unit to kilometer
!
!
C43  IS3D_full_switch  IS3D_grid_format  IS3D_surf_switch
C43  IS3D_format      IS3D_binary      IS3D_File_freq
C43  IS3D_time        IS3D_frequency   IS3D_start      IS3D_end      IS3D_unit_conv
!    0                1                0                ! dt = 60
!    1                0                14400             ! every 10 day simulation
!    4                60               0                999999       0.001 ! every 1 hr
!    0                1                0                ! dt = 90
!    1                0                9600             ! every 10 day simulation
!    4                40               52800            999999       0.001 ! every 1 hr; from 55day
!    0                1                0                ! dt = 150
!    1                0                5760             ! every 10 day simulation
!    4                24               0                999999       0.001 ! every 1 hr; from 55day
!    0                1                0                ! dt = 300
!    1                0                2880             ! every 10 day simulation
!    4                12               0                999999       0.001 ! every 1 hr
!    0                1                0                ! dt = 600
!    1                0                144              ! every 1 day simulation
!    4                6                0                999999       0.001 ! every 1 hr; from 50day
!-----!
C43_1 3D output file (tec3D_###.dat or vtk3D_###.dat) variable selection
!    It will be used only if (IS3D_full_switch == 1 or IS3D_surf_switch == 1), so do not deactivate this card.
!
!    variable:      [i]  dummy variable id (do not change order)
!    IS3D_variable: [i]  0/1  off/on option for each variable
!
C43_1 variable      IS3D_variable(:)
!    u              1
!    v              1
!    w              1
!    salt           1
!    temp           1
!    rho            0
=====!

```

Figure 3-24. Card Image C43 & C43_1.

Same as C42. If you activate **IS3D_surf_switch**, you will have not a full 3D view but only surface fluctuations.

```

=====!
C44  2D Dump file: dump2D_****.dat
!    Vertically averaged values of [eta, u, v, salt, temp, rho] at nodes will be written
!
!    IS2D_dump_switch:  [i]  0/1  off/on option for 'dump2D_****.dat'
!    IS2D_dump_binary:  [i]  Select file output format: ASCII or Binary
!                           0 = ASCII
!                           1 = Binary
!    IS2D_dump_time:    [i]  output file time unit
!                           1 = second, 2 = minute, 3 = hour, 4 = day
!    IS2D_dump_File_freq: [i] 2D dump file generation frequency
!                           A new file generation every [dt * IS2D_dump_File_freq] times
!    IS2D_dump_frequency: [i] Output dump frequency: printout time = dt*IS2D_dump_frequency [sec]
!    IS2D_dump_start:    [i] 2D dump start time (simulation step)
!    IS2D_dump_end:      [i] 2D dump end time (simulation step)
!
C44  IS2D_dump_switch      IS2D_dump_binary      IS2D_dump_time      IS2D_dump_File_freq
C44  IS2D_dump_frequency  IS2D_dump_start      IS2D_dump_end
      0                    0                      4                144      ! generate a file every day
      6                    1                      999999          ! dump every hr
=====!
C45  3D Dump file: dump3D_****.dat
!    eta:                at node & surface only,
!    u, v, w:            at node & at vertical level [z0 ~ zn]
!    salt, temp, rho:    at node & at vertical layer [z1 ~ zn]; mid-position
!
!    IS3D_dump_switch:  [i]  0/1  off/on option for 'dump3D_****.dat'
!    IS3D_dump_binary:  [i]  Select file output format: ASCII or Binary
!                           0 = ASCII
!                           1 = Binary
!    IS3D_dump_time:    [i]  output file time unit
!                           1 = second, 2 = minute, 3 = hour, 4 = day
!    IS3D_dump_File_freq: [i] 3D dump file generation frequency
!                           A new file generation every [dt * IS3D_dump_File_freq] times
!    IS3D_dump_frequency: [i] Output dump frequency: printout time = dt*IS3D_dump_frequency [sec]
!    IS3D_dump_start:    [i] 3D dump start time (simulation step)
!    IS3D_dump_end:      [i] 3D dump end time (simulation step)
!
C45  IS3D_dump_switch      IS3D_dump_binary      IS3D_dump_time      IS3D_dump_File_freq
C45  IS3D_dump_frequency  IS3D_dump_start      IS3D_dump_end
      0                    1                      4                14400     ! generate a file every 10day
      120                  1                      999999          ! dump every hr
=====!

```

Figure 3-25. Card Image C44 & C45.

GOM also provides 2D and 3D out dump options with either ASCII or binary output options, and in this case, you can easily construct any visualization tool-type data structure from the dump output file. Note that you will get the maximum simulation speed with the binary output options, thus these options will be beneficial especially for a large project.


```

=====!
C46  NetCDF dump file:
!
!   netcdf_switch: [i]  0/1  off/on option for 'netcdf_***.nc'
!   netcdf_File_freq: [i] NetCDF out file generation frequency
!   A new file generation every [dt * Netcdf_File_freq] times
!   netcdf_frequency: [i] NetCDF output data frequency: output time = dt*Netcdf_frequency [sec]
!   netcdf_start:      [i] NetCDF data start time (simulation step)
!   netcdf_end:        [i] NetCDF data end time (simulation step)
!
C46  netcdf_switch  netcdf_File_freq  netcdf_frequency  netcdf_start  netcdf_end
!   1                288                12                1                999999 ! dt = 300 / every 1day simulation /every 1 hr
!   1                24                 12                1                999999 ! dt = 300 / every 2hr simulation /every 1 hr
! -----!
C46_1 NetCDF output variable selection at node
!   It will be used only if (Netcdf_switch == 1), so do not deactivate this card.
!
!   netcdf_variable_node: [i]  0/1  off/on option for each variable
!   eta_node:             (maxnod)  water surface elevation at node [m]
!   u_node:               (0:maxlayer,maxnod) velocity u (true east/west) at node (at each vertical level) [m/s]
!   v_node:               (0:maxlayer,maxnod) velocity v (true north/south) at node (at each vertical level) [m/s]
!   w_node:               (0:maxlayer,maxnod) velocity w at node (at each vertical level) [m/s]
!   salt_node:            (maxlayer,maxnod) salinity at horizontal nodal point at each vertical layer [psu]
!   temp_node:            (maxlayer,maxnod) temperature at node [C]
!   rho_node:             (maxlayer,maxnod) rho at node [kg/m3]
!   wind_u_at_node:      (maxnod)    wind speed in x-direction at each node [m/s]
!   wind_v_at_node:      (maxnod)    wind speed in y-direction at each node [m/s]
!   airp_at_node:        (maxnod)    air pressure at node [Millibars]
!   ubar_node:           (maxnod)    vertically averaged velocity u (true east/west) at node [m/s]
!   vbar_node:           (maxnod)    vertically averaged velocity v (true north/south) at node [m/s]
!   sbar_node:           (maxnod)    vertically averaged salinity at node [psu]
!   tbar_node:           (maxnod)    vertically averaged temperature at node [C]
!   rbar_node:           (maxnod)    vertically averaged water density (rho) at node [kg/m3]
!
C46_1 variable          netcdf_variable_node(:)
eta_node                1
u_node                  1
v_node                  1
w_node                  1
salt_node               0
temp_node               0
rho_node                0
wind_u_at_node          0
wind_v_at_node          0
airp_at_node            0
ubar_node               0
vbar_node               0
sbar_node               0
tbar_node               0
rbar_node               0
! -----!
C46_2 NetCDF output variable selection at cell center
!   It will be used only if (Netcdf_switch == 1), so do not deactivate this card.
!
!   netcdf_variable_cell: [i]  0/1  off/on option for each variable
!   eta_cell:             (maxele)  water surface elevation at cell center [m]
!   wn_cell:              (0:maxlayer,maxele) velocity w at prism face [m/s]
!   salt_cell:            (maxlayer,maxele) salinity at cell center [psu]
!   temp_cell:            (maxlayer,maxele) temperature at cell center [C]
!   rho_cell:             (maxlayer,maxele) rho at element center [kg/m3]
!   Kv:                   (0:maxlayer,maxele) vertical eddy diffusivity at cell [m2/s]
!
C46_2 variable          netcdf_variable_cell(:)
eta_cell                0
wn_cell                 0
salt_cell               0
temp_cell               0
rho_cell                0
Kv                      0
! -----!
C46_3 NetCDF output variable selection at face
!   It will be used only if (Netcdf_switch == 1), so do not deactivate this card.
!
!   netcdf_variable_face: [i]  0/1  off/on option for each variable
!   Av:                   (0:maxlayer,maxface) vertical eddy viscosity [m2/s]
!   Kh:                   (maxlayer,maxface) Horizontal diffusivity [m2/s]
!
C46_3 variable          netcdf_variable_face(:)
Av                      1
Kh                      0
! -----!

```

Figure 3-26. Card Image C46, C46_1, C46_2, & C46_3

The last output options are about the NetCDF file format; nowadays, the use of the NetCDF file format is almost inevitable in numerical ocean modeling field. You can select multiple output variable options either at cell center, node, or even at the cell face. The variable explanation parts in each card show the variable array information. Here, if the variable array dimension is (maxlayer,**), then the variable is located at each vertical “**layer**”. If the variable array dimension is (0:maxlayer,**), then the variable is located at each vertical “**level**”. See more details about the vertical “layer” and “level” in Figure 3-7. Note that do not turn off the lines in C46_1, C46_2, and C46_3 even though you are not printing out the netcdf files, i.e., **netcdf_switch = 0**.

```

!=====!
C47  Write diagnostic files for important subroutines
!
!   dia_advection:      [i]   0/1   off/on for 'solve_nonlinear_advection.f90'
!   dia_momentum:      [i]   0/1   off/on for 'solve_momentum_equation.f90'
!   dia_freesurface    [i]   0/1   off/on for 'solve_freesurface_equation.f90'
!   dia_eta_at_ob      [i]   0/1   off/on in  'solve_freesurface_equation.f90'
!   dia_bottom_friction [i]   0/1   off/on for 'calculate_bottom_friction.f90'
!   dia_face_velocity  [i]   0/1   off/on in  'calculate_horizontal_velocities.f90 & calculate_vertical_velocities.f90'
!   dia_node_velocity  [i]   0/1   off/on in  'calculate_velocity_at_node.f90'
!
C47  dia_advection      dia_momentum      dia_freesurface      dia_eta_at_ob
C47  dia_bottom_friction dia_face_velocity dia_node_velocity
      0                  0                  0                  0
      0                  0                  0
!=====!
! End of file =====!

```

Figure 3-27. Card Image C47.

The last Card Image C47 controls the diagnostic files for some important subroutines. The diagnostic files provide more detailed information at each time step.

4 Additional Input Files

The GOM uses several additional input files in addition to three essential input files which are explained in the previous section.

4.1 harmonic_ser.inp

```
!C harmonic_ser.inp
!C
!C harmonic_ser_id_dummy: [i] ID number of tidal (harmonic) elevation time series. Dummy variable
!C no_tidal_constituent: [i] Total number of tidal constituent in each series. This number should not need to be identical
!C tidal_phase_shift: [r] Tidal phase shift in [degree].
!C To use cosine wave: 0.0
!C To use sine wave: 90.0
!C tidal_period: [r] Tidal period in [hour] at current location (boundary node)
!C tidal_amplitude: [r] Tidal amplitude at current location (boundary node)
!C tidal_phase: [r] Tidal phase in [degree] at current location (boundary node)
!C tidal_nodal_factor: [r] Nodal factor, f, at the reference time (model simulation begin time)
!C This should be identical for same tidal constituent.
!C Should be close to 1.0
!C equilibrium_argument: [r] Earth equilibrium argument (phase at Greenwich) in [degree] at the reference time (model simulation begin time)
!C This should be identical for same tidal constituent.
!C forcing_name: [c] Forcing symbol (dummy variable, just for the reference, will not be read)
!C
!C harmonic_ser_id_dummy no_tidal_constituent tidal_phase_shift
!C tidal_period tidal_amplitude tidal_phase tidal_nodal_factor equilibrium_argument forcing_name
1 1 0.0
12.42 0.3048 0.0 1.0 0.0 ! M2
```

Figure 4-1. harmonic_ser.inp.

The input file *harmonic_ser.inp* specifies the harmonic tidal elevation variation at the specified boundary cells. Periodic tidal boundaries can be included setting **num_tide_bc** > 0 (in C20). If **num_tide_bc** is greater than 0, you should specify how many harmonic tidal series are involved as setting **num_harmonic_ser** (in C20). If these two variables are specified (and of course **num_ob_cell** > 0), harmonic tidal series file, *harmonic_ser.inp*, should be provided. The *harmonic_ser.inp* requires the number of **num_harmonic_ser** harmonic tidal information sets. Note that most of the rest time-series input files have a similar structure.

4.2 eta_ser.inp

```
!C eta_ser.inp:
!C
!C eta_ser_id_dummy [i] ID number of surface elevation (eta) time series. Dummy variable
!C eta_ser_data_num [i] Number of time data lines at each time series
!C eta_ser_time_conv [r] Time unit conversion factor to [sec]
!C eta_ser_time_adjust [r] Additive adjustment to time values
!C eta_ser_unit_conv [r] Surface elevation (eta) unit conversion factor to [m]
!C eta_ser_adjust [r] Additive adjustment to eta values
!C eta_ser_time: [r] Surface elevation data time
!C eta_ser_eta: [r] eta (surface elevation) in [m]
!C
!C eta_ser_id_dummy eta_ser_data_num eta_ser_time_conv eta_ser_time_adjust eta_ser_unit_conv eta_ser_adjust
!C eta_ser_time eta_ser_eta: (1:eta_ser_data_num,1:num_ob_cell)
1 3 3600.0 0.0 1.0 0.0
0.00000 0.00
1.00000 0.05
9999.00000 0.05
2 3 3600.0 0.0 1.0 0.0
0.00000 0.00
1.00000 -0.05
9999.00000 -0.05
```

Figure 4-2. eta_ser.inp

The input file *eta_ser.inp* specifies the water surface elevation variation at the specified boundaries. If **num_eta_bc** (in C20) is greater than 0, *eta_ser.inp* is required. Like *harmonic_ser.inp*, *eta_ser.inp* requires the total number of **num_eta_ser** data sets, and each data set consists of a header line and **eta_ser_data_num** lines.

4.3 q_ser.inp

```
!C q_ser.inp:
!C
!C q_ser_id_dummy [i] ID number of river (Q) time series. Dummy variable
!C q_data_num [i] Total number data points in each series
!C q_time_unit_conv [r] Time unit conversion factor to [sec]
!C q_time_adjust [r] Additive adjustment to time values
!C q_unit_conv [r] Q unit conversion factor to [m3/s]
!C q_adjust [r] Additive adjustment to q values
!C q_interp_method [i] Choose one of the following two interpolation methods
!C 1 Linear Interpolation
!C 2 Lagrange Interpolation
!C q_ser_time: [r] Time. This number should start from 0 for correct interpolation.
!C q_ser_Q: [r] Q (discharge rate)
!C
!C q_ser_id_dummy q_data_num(:) q_time_unit_conv q_time_adjust q_unit_conv q_adjust q_interp_method
!C q_ser_time q_ser_Q
!C 1 2 86400.0 0.0 1.0 0.0 1
!C 0.0 10.0 10.0 10.0 10.0 10.0
!C 999999.0 10.0 10.0 10.0 10.0 10.0
!C 2 2 86400.0 0.0 1.0 0.0 1
!C 0.0 -10.0 -10.0 -10.0 -10.0 -10.0
!C 999999.0 -10.0 -10.0 -10.0 -10.0 -10.0
```

Figure 4-3. q_ser.inp

The input file *q_ser.inp* specifies river inflow or withdraws from specified boundary cells at each vertical layer. The file structure and concepts of variables in *q_ser.inp* are identical to *eta_ser.inp*.

4.4 windp_ser.inp

```
!C windp_ser.inp:
!C
!C   windp_ser_id      [i]   ID number of wind time series. Dummy variable
!C   windp_ser_data_num [i]   Total number of data points in each station
!C   windp_time_conv   [r]   Time unit conversion factor to [sec]
!C   windp_time_adjust [r]   Additive adjustment to time values
!C   windp_w_unit_conv [r]   Wind speed unit conversion factor to [m/s], or you can use this as an amplification factor
!C   windp_p_unit_conv [r]   Pressure unit conversion factor to [Millibar], or you can use this as an amplification factor
!C   windp_station_node [i]   Closest node number for this station
!C   windp_station_name [s]   dummy variable
!C   windp_ser_time    [r]   Julian Time started from the beginning of the simulation start year.
!C   windp_u           [r]   Wind speed in x-direction [m/s]; note that this unit is fixed.
!C   windp_v           [r]   Wind speed in y-direction [m/s]; note that this unit is fixed.
!C   windp_p           [r]   Pressure in [Millibar]; note that this unit is fixed.
!C
!C windp_ser_id      windp_ser_data_num  windp_time_conv  windp_time_adjust  windp_w_unit_conv  windp_p_unit_conv  windp_station_node  windp_station_name(i)
!C   windp_ser_time  windp_u            windp_v          windp_p
!C   1                744                86400.00000      0.00000            1.0                1.0                164802              !
!C   0.00000          7.15104            8.22633         1004.80000
!C   0.04167          10.11311           8.79119         1004.80000
!C   0.08333          9.98294             7.52269         1004.80000
!C   0.12500          11.66008            8.78650         1004.90000
!C   0.16667          11.02620            7.43727         1004.80000
```

Figure 4-4. windp_ser.inp.

The file *windp_ser.inp* specifies measured wind and atmospheric pressure information. The file structure and concepts of variables in *wind_ser.inp* are identical to *eta_ser.inp*. Maximum three wind and atmospheric pressure (windp) stations are allowed in this file. If you have more than three stations, consider using wind model data with *hurricane_ser.inp*, in which you have to provide wind and atmospheric pressure information at every model grid node.

4.5 hurricane_ser.inp

```
!C hurricane_ser.inp
!C Real-time hurricane data from NOAA/HRD
!C
!C   serial_num:      [i]   dummy number for the node number
!C   wind_u:         [r]   Wind velocity component [m/s]: x-direction
!C   wind_v:         [r]   Wind velocity component [m/s]: y-direction
!C   air_p           [r]   Atmospheric pressure in [millibar]
!C
!C hurricane_start_jday hurricane_end_jday hurricane_dt
!C serial_num wind_u wind_v air_p
      1      0.06   6.51  1015.0647 ! the first time set start for maxnod (=126474)
      2      0.06   6.51  1015.0641
      3      0.05   6.51  1015.0635
      .      .      .      .
      .      .      .      .
      .      .      .      .
126472     -1.09   6.94  1013.9925
126473     -1.09   6.94  1013.9908
126474     -1.09   6.94  1013.9916 ! end of the first time set
      1      0.98   6.10  1016.4757 ! the second time set start for maxnod (=126474)
      2      0.98   6.10  1016.4756
      3      0.98   6.10  1016.4754
      .      .      .      .
      .      .      .      .
      .      .      .      .
126472      0.26   7.69  1015.6986
126473      0.26   7.69  1015.6976
126474      0.26   7.69  1015.6982 ! end of the second time set
```

Figure 4-5. hurricane_ser.inp.

The file hurricane_ser.inp may be a quite big file depending on the grid size. The data starts and end times are defined in C31_1, and the data interval between each data set is defined with **hurricane_dt**.

4.6 salt_init.inp & salt_ser.inp / temp_init.inp & temp_ser.inp

```
! salt_init.inp:
! initial salinity condition at each horizontal nodal and vertical center position at each layer
! If you want to add more comments, then put '!' at the first column of the line.
! node_id, (salt_node(i,k), k=1,maxlayer)
!
1  0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
2  0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
3  0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
.
.
.
.
.
61 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
62 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
63 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
```

Figure 4-6. salt_init.inp

```
!C salt_ser.inp:
!C
!C salt_ser_id_dummy: [i] ID number of salinity [psu] time series. Dummy variable
!C salt_data_num: [i] Total number data points in each series
!C salt_time_unit_conv: [r] Time unit conversion factor to [sec]
!C salt_time_adjust [r] Additive adjustment to time values
!C salt_unit_conv: [r] Salinity unit conversion factor to [kg/m3]
!C salt_adjust: [r] Additive adjustment to salinity values
!C salt_ser_time: [r] Time. This number should start from 0 for correct interpolation.
!C salt_ser_salt: [r] Salinity
!C
!C salt_ser_id_dummy salt_ser_data_num salt_time_conv salt_time_adjust salt_unit_conv salt_adjust
!C salt_ser_time salt_ser_salt
1 0.0 30.0 2 1.0 0.0 1.0 0.0
999999.0 30.0
```

Figure 4-7. salt_ser.inp

Initial salinity and temperature data are provided with *salt.inp* and *temp.inp*, respectively, at the layer of each node. The boundary time series files are *salt_ser.inp* and *temp_ser.inp*, respectively.

4.7 Obc_info.inp & Qbc_info.inp

```

! Obc_info.inp; this is identical to C20_1
C20_1 Eta boundary information
!   Activate if num_ob_cell > 0.
!   Should have 'num_ob_cell' information (lines).
!
!   serial_num      [i]   Station number (dummy number)
!   ob_nodes(i,1)   [i]   First node of the station (or the element, in counterclockwise direction)
!   ob_nodes(i,2)   [i]   Second node of the station (or the element, in counterclockwise direction)
!   ob_eta_type     [i]   Open boundary surface elevation type:
!                           -1 Radiation bc
!                           No input is required; elevations will be computed as average of surrounding elevations.
!                           1 This boundary is forced by time history of elevation. eta_ser.inp is required
!                           2 This boundary is forced by tidal harmonic constituents; harmonic_ser.inp is required; default = c
!   harmonic_ser_id [i]   ID number of harmonic series at this station.
!                           If ob_eta_type == 2 or 3, it will be activated.
!                           It should be greater than 0.
!   eta_ser_id      [i]   ID number of surface elevation time series at this station.
!   salt_ser_id     [i]   ID number of salinity time series at this station.
!   temp_ser_id     [i]   ID number of temperature time series at this station.
!   ob_name         [c]   dummy variable. This will not be read.
!
C20_1 serial_num ob_nodes(i,1) ob_nodes(i,2) ob_eta_type(i) harmonic_ser_id(:) eta_ser_id(:) salt_ser_id temp_ser_id ob_name
      1          4295         4294           1           0           1           1           0           ! Pass
      2          4294         4127           1           0           1           1           0
      .           .           .           .           .           .           .           .
      .           .           .           .           .           .           .           .
      .           .           .           .           .           .           .           .
      69          50          51           1           0           2           2           0
      70          51          52           1           0           2           2           0

```

Figure 4-8. Obc_info.inp

5 Downloading, Compiling, and Executing the Code

To download the GOM, go to (of course I believe you already did):

- https://github.com/GOM-proj/GOM_release
- Click the blue “Code” button on the right side of the window
- Click “Download ZIP”

Source codes, which are all written in standard FORTRAN 90, are stored in the ‘*source*’ directory. The three of the most important source codes that you must take a look at are (1) *mod_global_variables.f90*, (2) *mod_file_definition.f90*, and (3) *main.f90*. The global variables and parameters for GOM are defined in *mod_global_variables.f90*. If you want to take a look at input and output files and folder structure or want to change input and output file/folder names, *mod_file_definition.f90* is the source code you must take a look at. The straightforward simulation flow chart of the GOM can be found in the main program, *main.f90*.

5.1 Install Fortran Compiler & NetCDF library

To compile the source code, you need a Fortran 90 compiler, and there are several different compilers. Among them, I specifically prefer to use GFortran from the GNU project, and it is a free compiler. **Note that GOM has been developed and tested under GFortran, thus it is not guaranteed with other compilers.** If your institute provides a super computing system, you just need to use the resources. However, if you are individual user, you should install “gfortran” by yourself on your personal computer, and here is the brief instruction to get “gfortran” both on your Linux/Unix or Windows systems. In addition, from the GOM_Rv1.2.0, you should install NetCDF (network Common Data Form) libraries as well.

5.1.1 Linux & Unix

If you use Linux/Unix, get “gfortran” with, e.g. (either one of these):

```
sudo apt-get install gfortran
sudo apt install gfortran
```

Then, check if it is installed correctly with:

```
gfortran --version
```

If gfortran is successfully installed, you will see like this (Figure 5-1):

```
GNU Fortran (Ubuntu/Linaro 4.6.3-1ubuntu5) 4.6.3
Copyright (C) 2011 Free Software Foundation, Inc.

GNU Fortran comes with NO WARRANTY, to the extent permitted by law.
You may redistribute copies of GNU Fortran
under the terms of the GNU General Public License.
For more information about these matters, see the file named COPYING
```

Figure 5-1. gfortran version check result.

Now, let's install NetCDF Fortran library. You will need to install the NetCDF C library and then the Fortran libraries as well. Open the terminal and check if NetCDF is installed as shown below:

```
nc-config --version
```

If you do not have the NetCDF library yet, you will see something like this (Figure 5-2).

```
jungwoo@jungwoo-VirtualBox:~$ nc-config --version
Command 'nc-config' not found, but can be installed with:
sudo apt install libnetcdf-dev
jungwoo@jungwoo-VirtualBox:~$
```

Figure 5-2. NetCDF install check.

Now, let's install the NetCDF libraries step by step: (1) update package lists, (2) install the NetCDF C library, (3) install the NetCDF Fortran library:

```
sudo apt update
sudo apt install libnetcdf-dev
sudo apt install libnetcdf-fortran-dev
```

Let's check if you successfully install the NetCDF libraries or not:

```
nc-config --version
```

Then, you will see the NetCDF library version as shown below (Figure 5-3):

```
jungwoo@jungwoo-VirtualBox:~$ nc-config --version
netCDF 4.9.2
```

Figure 5-3. NetCDF library install check.

Now, let's double check if the NetCDF library is successfully installed; you need this information when compiling the GOM (note here I am using "nf-config" not "nc-config" to make sure the NetCDF fortran library is installed):

```
nf-config --all
```

```
jungwoo@jungwoo-VirtualBox:~$ nf-config --all
This netCDF-Fortran 4.5.4 has been built with the following features:

--cc      -> gcc
--cflags   -> -I/usr/include -Wdate-time -D_FORTIFY_SOURCE=3
--fc       -> gfortran
--fflags   -> -I/usr/include -I/usr/include
--flibs    -> -L/usr/lib/x86_64-linux-gnu -lnetcdff -Wl,-Bsymbolic-functions -flto=auto -ffat-lto-objects -Wl,-z,relro -Wl,-z,now -lnetcdf -lnetcdf -lm
--has-f90   ->
--has-f03   -> yes

--has-nc2   -> yes
--has-nc4   -> yes

--prefix    -> /usr
--includedir -> /usr/include
--version    -> netCDF-Fortran 4.5.4

jungwoo@jungwoo-VirtualBox:~$
```

Figure 5-4. NetCDF Fortran library install check.

In the above figure (Figure 5-4), the red box information will be required to correctly link to the NetCDF libraries in the compiling process (see Section 5.2 to how to use this information in *makefile*).

5.1.2 Windows

If you are a Windows user, I recommend you download one of the following software packages:

- MinGW (Minimalist GNU for Windows)
 - <https://www.mingw-w64.org/>
 - You may be able to install MinGW64 directly from the website
 - MinGW is included in Cygwin and MSYS2, so you may want to install one of these, and the website (<https://www.mingw-w64.org/>) includes the link to these two distros.
- Cygwin (www.cygwin.com), which is a Unix-like environment and command-line interface for Microsoft Windows
- MSYS2 (<https://www.msys2.org/>), which is a software distro and building platform for Windows, and it can be an alternative of Cygwin.

After installing either MinGW, Cygwin, or MSYS2, check if it is installed correctly with the method I explained above. My suggestion is to install “MSYS2” since it is easier to install than Cygwin. The NetCDF library install process is similar to the installation process on the Linux as explained in the previous section (in Linux & Unix section), so google it and follow the instructions.

5.1.3 MSYS2 install guide

Now, I briefly explain how to install MSYS2 (since this is my preferred package among other packages listed above)

and some required libraries.

Install MSYS2:

- go and find the latest version (x86_64 version with exe, mostly at the bottom) from https://repo.msys2.org/distrib/x86_64/

Install gfortran:

- https://packages.msys2.org/packages/mingw-w64-x86_64-gcc-fortran
- `pacman -S mingw-w64-x86_64-gcc-fortran`
 - this will install gfortran on MINGW64
- `pacman -S mingw-w64-ucrt-x86_64-gcc-fortran`
 - this will install gfortran on UCRT64

Install make:

- <https://packages.msys2.org/packages/make>
 - ♦ this will install make on both UCRT64 & MINGW64

Install gdb (this is for debugging):

- https://packages.msys2.org/packages/mingw-w64-x86_64-gdb
- `pacman -S mingw-w64-x86_64-gdb`
- `pacman -S mingw-w64-x86_64-gdb-multiarch`

Let's check installed version:

- `$make --version`
- `$gcc --version`
- `$gfortran --version`
- `$gdb --version`

Once you install MSYS2, you will see three different consoles:

- MSYS2 MSYS:
 - Provides a Unix-like shell environment within Windows, allowing you to run familiar Linux commands like "ls", "cd", "make".
 - Primarily used for building cross-platform software that needs a POSIX-compliant environment.
 - Not typically used for directly compiling native Windows applications.
- MSYS2 UCRT64:

- A build environment within MSYS2 that utilizes the "Universal C Runtime" (UCRT) libraries from Microsoft.
- Considered the preferred choice for building most modern Windows applications due to its compatibility with newer Windows features.
- Access to UCRT libraries is included in the environment path.
- MSYS2 Mingw64:
 - Another build environment in MSYS2 for compiling native Windows applications, but uses older Windows system libraries compared to UCRT64.
 - May be necessary for older projects that specifically require the older Windows libraries.

Among these three console options, I used to use “MSYS2 Mingw64” console. So, you need to install the NetCDF libraries on that console environment. Let’s install NetCDF libraries on “MSYS2 Mingw64”. Open MSYS2 Mingw64 console and follow the instructions below:

- Install NetCDF library:
 - `pacman -S mingw-w64-x86_64-netcdf`
- Install NetCDF Fortran library:
 - `pacman -S mingw-w64-x86_64-netcdf-fortran`

Now, let’s check if it is successfully installed (note, here I am using “nc-config” not “nf-config” since “nf-config” is not a standalone package directly available for installation via “pacman” in MSYS2/MinGW64):

```
nc-config --all
```

Then, you will see something like below (Figure 5-5):

```

Jungwoo@DESKTOP-STH6NUG MINGW64 ~
$ nc-config --all
This netCDF 4.9.3 has been built with the following features:
--cc             -> /mingw64/bin/gcc.exe
--cflags         -> -I/mingw64/include
--libs           -> -L/mingw64/lib -lnetcdf
--static         -> -lHDF5::HDF5 -lHDF5::hdf5_hl -lz -lzip -lbz2 -lsz -lCURL::libcurl -lxml2

--has-dap        -> yes
--has-dap2       -> yes
--has-dap4       -> yes
--has-nc2        -> yes
--has-nc4        -> yes
--has-hdf5       -> yes
--has-hdf4       -> no
--has-logging    -> yes
--has-pnetcdf    -> no
--has-szlib      -> yes
--has-cdf5       -> yes
--has-parallel4  -> no
--has-parallel   -> no
--has-nczarr     -> yes
--has-zstd       -> yes
--has-benchmarks -> yes
--has-multifilers -> yes
--has-stdfilters -> deflate szip blosc zstd bz2
--has-quantize   -> yes

--prefix         -> /mingw64
--includedir     -> /mingw64/include
--libdir         -> /mingw64/lib
--pluginindir    -> /mingw64/lib/netcdf
--plugin-searchpath -> /mingw64/lib/netcdf;C:/ProgramData/hdf5/lib/plugin
--version        -> netCDF 4.9.3
--build-system   -> cmake

```

Figure 5-5 NetCDF Fortran library install check on MSYS2 Mingw64.

As previous, in the above figure (Figure 5-5), the highlighted box information will be required to correctly link to the NetCDF libraries in the compiling process (see Section 5.2 to how to use this information in *makefile*).

5.2 Compile Source codes

To store source codes and object files in separate folders, the *makefile*, which describes the relationships among source codes and provides commands for updating each file, is located in *'source/release'*. Before you compile the source code, you have to correctly set the NetCDF library paths. So, open *makefile*, then update the following two lines: **LD_LIBRARY_PATH** & **FCFLAGS**. You have to insert the red box information in Figure 5-4 (in the following figure, Figure 5-6, I am showing the NetCDF library path in my MSYS2).

```

# Libraries needed for linking -----#
# There should be no space between LD_LIBRARY_PATH and "=" and directory_name
LD_LIBRARY_PATH=-L/mingw64/lib -lnetcdff -lnetcdf

# flags for all (e.g., look for system .mod files, required in gfortran)
FCFLAGS += -I/mingw64/include

```

Figure 5-6. NetCDF library path update in the Makefile.

Once you update the NetCDF library path in the *makefile*, let's compile source codes:

- type *'make all'* on the command line, then the *makefile* will compile only newer source codes compared to the corresponding object files (*.o) and link object files, and finally make an executable file as defined in the *makefile*, e.g., *run_release.exe*.

- If there are no object files in the folder, the command '**make all**' will compile all source codes and generate an executable file.
- The command '**make clean**' will remove (or clean) all the object files and an executable file.
 - Note that if you are using a pure MinGW (i.e., not Cygwin or MSYS2), and thus if you use window's command window terminal or Windows Powershell, you have to change the very last line in **makefile** as below

This is the original setting:

clean:

```
rm -f *.O *.mod *.MOD *.exe
```

Now, you have to change this to ("rm" to "del"):

clean:

```
del -f *.O *.mod *.MOD *.exe
```

- Once you get the executable **run_release.exe**, You should copy the executable file into the model project folder in which actual simulation will be done.

The executable file, **run_release.exe**, should be located under the project folder with '**input**' and '**output**' folder together, i.e., your project folder must have the following structure:

- `/project_folder/input`
- `./project_folder/output`
- `./project_folder/run_release.exe`

If the OpenMP option is selected in the **makefile**, you may need to set the total core numbers with the environmental variable, OMP_NUM_THREADS. Then, type '**./run_release.exe**' on the command window to run the GOM. The following shows the commands I explained above:

```
$make clean
```

```
$make all
```

```
$cp run_release.exe <your_project_folder>
```

```
$export OMP_NUM_THREADS=8 (if it is compiled with OpenMP option and you want to use 8 cores.)
```

```
$./run_release.exe
```

Now you will see the GOM simulation progress on the terminal window.

6 Source Codes and Subroutines

The current version of the GOM (GOM_Rv1.0.0) consists of more than one hundred standard FORTRAN source files including one main program and three modules. Names and brief descriptions of source codes are shown in Table 6.1.

Table 6.1. Subroutines in GOM. (not yet finished)

Subroutine	Description
------------	-------------

7 References

- Casulli, V., Walters, R.A., 2000. An unstructured grid, three-dimensional model based on the shallow water equations. *International Journal for Numerical Methods in Fluids* 32, 331–348. [https://doi.org/10.1002/\(SICI\)1097-0363\(20000215\)32:3<331::AID-FLD941>3.0.CO;2-C](https://doi.org/10.1002/(SICI)1097-0363(20000215)32:3<331::AID-FLD941>3.0.CO;2-C)
- Casulli, V., Zanolli, P., 2005. High resolution methods for multidimensional advection–diffusion problems in free-surface hydrodynamics. *Ocean Modelling, The Second International Workshop on Unstructured Mesh Numerical Modelling of Coastal, Shelf and Ocean Flows* 10, 137–151. <https://doi.org/10.1016/j.ocemod.2004.06.007>
- Holland, G.J., 1980. An Analytic Model of the Wind and Pressure Profiles in Hurricanes. *Mon. Wea. Rev.* 108, 1212–1218. [https://doi.org/10.1175/1520-0493\(1980\)108<1212:AAMOTW>2.0.CO;2](https://doi.org/10.1175/1520-0493(1980)108<1212:AAMOTW>2.0.CO;2)
- Ji, Z.-G., 2017. *Hydrodynamics and Water Quality: Modeling Rivers, Lakes, and Estuaries*, 2nd edition. ed. Wiley, Hoboken, NJ.
- Kincaid, D.R., Respass, J.R., Young, D.M., 2017. ITPACK 2C: A FORTRAN Package for Solving Large Sparse Linear Systems by Adaptive Accelerated Iterative Methods.
- Kincaid, D.R., Respass, J.R., Young, D.M., Grimes, R.R., 1982. Algorithm 586: ITPACK 2C: A FORTRAN Package for Solving Large Sparse Linear Systems by Adaptive Accelerated Iterative Methods. *ACM Trans. Math. Softw.* 8, 302–322. <https://doi.org/10.1145/356004.356009>
- Lee, Jun, Lee, Jungwoo, Yun, S.-L., Kim, S.-K., 2020. Three-Dimensional Unstructured Grid Finite-Volume Model for Coastal and Estuarine Circulation and Its Application. *Water* 12, 2752. <https://doi.org/10.3390/w12102752>
- Smagorinsky, J., 1963. GENERAL CIRCULATION EXPERIMENTS WITH THE PRIMITIVE EQUATIONS. I. THE BASIC EXPERIMENT. *Mon. Wea. Rev.* 91, 99–164. [https://doi.org/10.1175/1520-0493\(1963\)091<0099:GCEWTP>2.3.CO;2](https://doi.org/10.1175/1520-0493(1963)091<0099:GCEWTP>2.3.CO;2)